HAETAE-2 Jasmin Implementation Verification

Current Proof Frontier for Mathematicians and Cryptographers

HAETAE top-down EasyCrypt working document

Status as of August 13, 2026

Abstract

This document reconstructs as mathematical statements the connections currently established between the HAETAE-2 paper specification, Jasmin implementation, and EasyCrypt proofs. The main results are a partial-correctness theorem that the first 992 bytes of the secret key returned by every terminating internal KeyGen execution equal the verification key, and relational theorems derived from this invariant: the actual generated Sign and Verify procedures produce the same 32-byte message-digest prefix and reach the same challenge-suffix state along their raw/internal paths. Statistical loss is 0 at this local boundary. Sequential VK/SK exports by the actual raw KeyGen caller, integer/64-bit address bindings, exact Sign/Verify traces, successful Verify's arrival at the tail, and hash-input bindings were also verified. The frame condition of actual Sign output, region-local hash equivalence, and the sequential trace of actual Sign followed by Verify were closed. However, the product/replay edge transporting stored μ observations from completed traces is frozen as an explicit paper boundary.

Along the signature functional path, the work proved the 1056-byte prefix-codec round trip under canonical inputs, HBZ prepare/apply, actual table certificates, pure rANS steps and normalization-byte stacks, actual suffix copying, and harness control. Week 10 added array/list suffix connections, actual W32/W64 word steps, byte/frame preservation and cursor non-underflow in the generated inner normalization loop, leaf theorems for final 4-byte serialization, and the actual `_rans_encode` successful size bound $4 \le 1024 - \mathsf{off} \le 1024$. Week 11 installed the tail-aware invariant, generated word-step bridge, one-symbol outer transition, and final serialization composition inside the actual generated procedure. The resulting success-conditioned partial-correctness theorem compiles: a terminating actual `_rans_encode` either returns failure or, on success, its entire returned suffix equals pure `trace_bytes`, with the offset–length equality and initial-prefix frame. Week 12 directly closed the actual generated `_rans_decode` initial 4-byte parsing, concrete table lookup and word update, 0/1/2-byte normalization replay, outer/inner loop invariants, and final rejection check. The resulting exact-trace partial-correctness theorem compiles: a terminating execution on exact `trace_bytes` input recovers 1024 symbols, consumes the exact size, preserves the output tail, and returns $bad = 0$. Week 13 transported the actual encoder suffix and the pointwise region equality established by actual copying into the decoder's exact-trace input, proving both decoder non-execution on failure and 1024-symbol recovery on success in the same harness. Week 14 lifted the actual full-HBZ prepare–rANS–apply boundary to the production SignaturePack/Unpack extraction boundaries, proving preservation of the initial buffer on failure and recovery of the original 1024 coefficients with a tail frame on success in one contract. Week 15 showed that actual preparation of all-zero HBZ produces an all-6 symbol sequence, and excluded actual encoder failure using a pure upper bound of 1020 bytes on that sequence's normalization budget. Thus the production pack/unpack nonzero-size round trip compiles for terminating fixed-input executions. This is Hoare partial correctness; it does not imply termination, losslessness, or probability-1 success.

Week 16's 82 authored targets freeze the actual KeyGen matrix/finalizer snapshot, actual three-helper Sign control, actual five-helper Verify sequence, V-1/V-2/V-5/V-6, the

W64 norm gate, and tail/mismatch word expressions at the `PAPER-FROZEN` boundary. In the frozen claim matrix, KeyGen is at `STOP-KG-NTT`, Sign at `STOP-SIGN-CHAL-MODE2`, and Verify at `STOP-VERIFY-MATRIX-CRT`. These decisions do not establish correctness of the three complete algorithms.

Post-freeze auxiliary proofs advanced the NTT/Rq boundary without retroactively extending the frozen paper. Odd-root/full-NTT spectral action, mathematical 2-by-3/2-by-4 row products, and preservation of addition, multiplication, and 3-term dot-product representation from `Rq.poly` to `HAETAE_Algebra.poly` were closed. The work also connected the nonces 515/516, SHAKE128 framing, little-endian 16-bit parsing, and < 64513 rejection semantics of terminating actual mode-2 `avec` expansion to the paper's a, and composed an independent paper qj term with the finalizer snapshot. However, compiled theorems and concrete traces confirmed that the existing security model's synthetic `haetae_mode23_qj_vector` is a different object from the paper's qj and actual expansion. The latest auxiliary decision is therefore `STOP-KG-AVEC-QJ`, with `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS` as the first repair obligation. The hint (h) codec remains deferred. Consequently, $\delta_{\mu,\text{raw_api_accept}} = 0$ and $\delta_{\text{Encoding}} = 0$ cannot yet be claimed. Suffix codecs, context, highbits/LSB, the complete challenge, retry distributions, and implementation-level EUF-CMA also remain open. This guide is dedicated to distinguishing these boundaries.

One-sentence summary

The current proofs close

$$\operatorname*{prefix}_{992}(sk) = vk \Rightarrow \operatorname*{take}_{32}(\mu^{64}_{\text{Sign}}) = \mu^{32}_{\text{Verify}} \Rightarrow \text{equal challenge-suffix states}$$

for actual generated raw/internal procedures, extending to actual raw-API exports, exact traces, Sign output frames, and region-local memory connections. Along the functional path, the actual signature-prefix codec is followed by closed HBZ prepare/apply, actual table certificates and pure rANS steps, a common normalization-byte trace, actual suffix copying, and 3-call harness control. Week 10 added actual encoder inner-write semantics and successful size bounds. Week 11 composed exact inner invariants preserving the pure tail, generated word updates, the outer loop, and the final four stores to close the actual encoder's successful-suffix refinement. Week 12 closed the actual decoder's forward consumption of the same pure trace, recovering the original symbols, exact consumed size, and output-tail frame. Week 13 composed the failure/success branches and complete symbol recovery on success in a harness directly calling the actual encoder–copy–decoder procedures. Week 14 composed that result with actual full-HBZ prepare/apply and transported it to the production SignaturePack/Unpack boundaries. Week 15 closed actual all-6 preparation, encoder-failure exclusion, and the production fixed-input round trip for all-zero HBZ. Week 16's 82 authored targets froze the actual KeyGen matrix/finalizer snapshot and the limited actual Sign/Verify control and machine-word boundaries as `PAPER-FROZEN`. The frozen matrix's stopping points are respectively `STOP-KG-NTT`, `STOP-SIGN-CHAL-MODE2`, and `STOP-VERIFY-MATRIX-CRT`.

The post-freeze auxiliary layer closed NTT spectral action, native Rq row products, the Rq–HAETAE representation bridge, the actual `avec` paper-a semantics, and the separate paper-qj term. Conversely, the security model's synthetic qj was proved unequal to these two objects, so the latest decision is `STOP-KG-AVEC-QJ`. This does not mark full KG-1–KG-4, $As = qj \pmod{2q}$, full KeyGen, the h codec, the complete signature codec, or Sign–Verify correctness as complete. The product/replay theorem connecting stored μ

observations from completed traces remains **DEFERRED**. Fixed-input termination/losslessness, `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS`, MINCORE Sign/Verify/composition, the (h) suffix, and the complete Verify gate remain open. Verify's pure NTT leaves are closed, but the actual 2-by-4 matrix-accumulation and CRT/freeze procedure leaves block the combined theorem. Promotion to API/encoding zero loss and EUF-CMA is therefore not yet justified.

Contents

1 Audience, scope, and status terminology **5**
- 1.1 What this guide makes easier to read . . . 5
- 1.2 Terminology conventions . . . 5
- 1.3 Status terms . . . 6
- 1.4 Snapshot and targets . . . 6

2 Mathematical targets at the paper level **7**
- 2.1 Minimal HAETAE-2 background . . . 7
- 2.2 Functional-correctness target for the mode-2 signature codec . . . 8
- 2.3 Mathematical structure of HBZ leaves, rANS single steps, and byte traces . . . 9
- 2.4 Week 10 mathematical boundary of the actual encoder . . . 10
- 2.5 Week 11 closure of the actual encoder trace . . . 11
- 2.6 Week 12 closure of the actual decoder trace . . . 11
- 2.7 Week 13 closure of actual rANS core composition . . . 12
- 2.8 Week 14 closure of production full-HBZ wrappers . . . 13
- 2.9 Week 15 actual success postcondition for fixed all-6 input . . . 13
- 2.10 Week 16 actual KeyGen matrix/finalizer snapshot . . . 14
- 2.11 Top-level functional-correctness targets . . . 15
- 2.12 Top-level security-composition target . . . 15

3 The formal language for reading statements: EasyCrypt **16**
- 3.1 Core predicates and relations . . . 16
- 3.2 Hoare statements and relational equivalence . . . 21
- 3.3 Exact traces and accepted tails . . . 21
- 3.4 Array capacity and logical length . . . 21

4 The proof chain closed so far **22**
- 4.1 Step 1: the packed-key prefix returned by KeyGen . . . 22
- 4.2 Step 2: from local arrays to Verify memory . . . 22
- 4.3 Step 3: from copy helpers to the actual raw KeyGen caller . . . 23
- 4.4 Step 4: actual raw Sign/Verify μ procedures . . . 24
- 4.5 Step 5: equality of the challenge suffix . . . 24
- 4.6 Step 6: exact traces of actual raw Sign/Verify callers . . . 24
- 4.7 Step 7: accepted-path generated adapters and the remaining gap . . . 25
- 4.8 Step 8: Sign-output frames and region-local sequential traces . . . 25
- 4.9 Step 9: the actual mode-2 signature prefix codec . . . 26
- 4.10 Step 10: actual HBZ leaves, table certificates, and pure rANS steps . . . 26
- 4.11 Step 11: rANS byte stacks, actual suffix copying, and harness control . . . 27
- 4.12 Step 12: actual encoder inner semantics and successful sizes . . . 28
- 4.13 Step 13: Week 11 actual encoder trace closure . . . 30
- 4.14 Step 14: Week 12 actual decoder semantic refinement . . . 31

4.15 Step 15: Week 13 actual rANS core inverse composition . . . 32
4.16 Step 16: Week 14 production full-HBZ wrapper composition . . . 33
4.17 Step 17: Week 15 actual success postcondition for fixed all-6 input . . . 34
4.18 Step 18: Week 16 MINCORE KeyGen snapshot . . . 35
4.19 Step 19: Week 16 KG-NTT-MUL audit and stop verdict . . . 36
4.20 Step 20: paper freeze and post-freeze KeyGen semantic alignment . . . 37

5 Exactly what has been established for security **38**
5.1 The meaning of local zero loss . . . 38
5.2 Current claim matrix . . . 39
5.3 Distinguishing assumptions, contracts, and proof obligations . . . 43

6 Remaining proof obligations and recommended order **43**
6.1 Actual API boundaries closed in Weeks 5/6 . . . 43
6.2 Frozen security gate: product/replay for stored μ . . . 44
6.3 Recently closed gates: actual rANS core and production full HBZ . . . 44
6.4 KeyGen gate: from frozen STOP-KG-NTT to current STOP-KG-AVEC-QJ . . . 45
6.5 Separate functional-correctness path . . . 46
6.6 Next gates: context and the full challenge transcript . . . 46
6.7 Subsequent functional and probabilistic obligations . . . 46
6.8 Dependency-based work order . . . 48

7 Trust boundaries and reproduction **49**
7.1 What is trusted . . . 49
7.2 Proof verification pipeline . . . 50
7.3 Building this document . . . 52

A Theorem and source index **52**

B Notation and abbreviations **63**

1 Audience, scope, and status terminology

1.1 What this guide makes easier to read

The existing `latex/` documents in this workspace are weekly research and development records. This guide reorganizes the material so that mathematicians and cryptographers can directly answer three questions.

1. Which mathematical equality in the paper becomes which byte invariant in the implementation?
2. What is the strongest theorem actually checked by EasyCrypt so far, and what are its premises?
3. What else is needed for that theorem to become complete functional correctness or an EUF-CMA theorem?

Actual EasyCrypt declarations are the primary evidence in this guide. `CLAIM_LEDGER.md` and `manifests/paper-artifacts.md` define the frozen state; post-freeze reports take precedence for subsequent results. Extraction manifests and weekly reports follow. The opening portions of weekly documents are historical snapshots, so the following materials take precedence when assessing the current state.

- `manifests/paper-artifacts.md`, `POST_FREEZE_KG_RQ_HAETAE_REPORT.md`, `POST_FREEZE_KG_ACTUAL_AVEC_QJ_REPORT.md`;
- `WEEK16_KG_NTT_MUL_REPORT.md`, the Week 16 KG-NTT audit note `latex/sections/33-week16-kg-ntt-mul-stop.tex`;
- `WEEK16_SIGN_REPORT.md`, `WEEK16_VERIFY_REPORT.md`, `logs/verify-all-week16-verify-matrix-crt.log`;
- `WEEK16_KG_REPORT.md`, `WEEK16_MINCORE_PLAN.md`, `logs/verify-all-week16-kg-first-task.log`;
- `WEEK15_REPORT.md`, `RANS_ACTUAL_SUCCESS_WITNESS.md`, the Week 15 research note `latex/sections/31-week15-rans-actual-success-witness.tex`;
- `WEEK14_REPORT.md`, the Week 14 research note `latex/sections/30-week14-hbz-full-wrapper-composition.tex`, `HBZ_FULL_WRAPPER_COMPOSITION.md`;
- `WEEK13_REPORT.md`, the Week 13 research note `latex/sections/29-week13-rans-core-composition.tex`, the independent verification record `logs/week13-independent-verifier.md`;
- `PRODUCT_REPLAY_DECISION.md`, `SIGNATURE_CODEC_OBLIGATIONS.md`, `MODE2_HBZ_CODEC_OBLIGATIONS.md`;
- `RANS_PROOF_DESIGN.md`, `RANS_BYTE_STACK_INVARIANT.md`, `RANS_ENCODER_INVARIANT.md`, `RANS_DECODER_INVARIANT.md`, and operational documents.

Line numbers change during editing, so the guide uses file paths and declaration names as stable identifiers.

1.2 Terminology conventions

The original Korean edition used the following conventions to help mathematicians and cryptographers match implementation terminology to the source. This English translation uses the corresponding English terms.

1. On first use, the Korean term preceded its English original in parentheses. Examples include refinement, invariant, trace, frame condition, prefix, and suffix.
2. Repeated uses in the same context preferred Korean, while sentences directly compared with code could repeat the previously introduced English term.
3. EasyCrypt theorem and procedure names, claim identifiers, filenames, and code snippets were left unchanged for exact lookup. Standard abbreviations such as `rANS`, `HBZ`, `W32`, `W64`, and `EUF-CMA` are also retained.
4. The terms "actual" for an actual generated procedure, "canonical," and "success-conditioned" were introduced with their Korean counterparts.

1.3 Status terms

PROVED The EasyCrypt theorem compiles without cached proofs under the stated premises. This does not mean that every premise of the parent goal is discharged.

PARTIAL Genuine subtheorems needed by the parent goal exist, but unresolved premises or call boundaries remain for the complete parent goal.

SPECIFIED The goal's type and logical formula compile, but no theorem proving it is yet available.

BLOCKED A required extraction, memory semantics, probability distribution, numerical-error bound, or cryptographic reduction is concretely missing, preventing the next composition.

DEFERRED The gap is precisely identified but is not closed under the currently permitted proof rules and scope, so it is frozen as an explicit paper boundary. This does not mean that it is proved or unnecessary.

Boundary statement 1.1 (Meaning of `PROVED`)**.** In this guide, **PROVED**always means that the stated theorem has been verified for executions satisfying its displayed premises. In particular, **PROVED**for a Hoare partial-correctness theorem does not imply termination, losslessness, or equality of output distributions.

1.4 Snapshot and targets

Boundary statement 1.2 (The frozen 82 targets and post-freeze auxiliary evidence)**.** After target 77, `Mode2KeygenNttMulBridge.ec`, target 78, `Mode2SignAcceptedCore.ec`, and target 82, `Mode2VerifyCoreSequence.ec`, freshly compile the actual Sign/Verify control boundaries. The completed 78-target run before the Verify changes is preserved in `logs/verify-all-before-week16-verify.log`. Preserved `logs/verify-all-before-paper-freeze.log`, with final line `RESULT PASS authored-targets=82 cache=-no-eco` and SHA-256 `4cd64e5a656be82710bca1410c4d19403a3c661d6b91b0319a0ea8f7c91646da`, is completed evidence immediately before the freeze. The `logs/verify-all-summary.txt` tracked at baseline commit `e87bcc3` includes the paper-freeze audit and has the same terminal result with SHA-256 `08ef9639dc73d56dba42d02999d07897d29bc8e60aa100a648e9437fd64387ad`. The three `post-freeze/KgActualAvecQj*.ec` targets are not included in these 82 and were each freshly compiled separately. **PROVED**for a Hoare theorem still does not imply actual procedure termination or probability-1 execution.

Table 1: Verification boundaries fixed by this guide

Item	Fixed target
Paper specification	`CryptoLab_HAETAE.pdf`, 46 pages, created 2026-02-04, SHA-256 `80d28b3af9af2a27dba0eb4746f9b575 5e05cb9a86bb6596f45fda06c605f3f6` [2]
Parameter set	HAETAE-2 only: $n = 256$, $(k, \ell) = (2, 4)$, $q = 64513$, $\eta = 1$, $\tau = 58$
Implementation	Pinned `haetae-ref-jasmin/` sources and `jasmin2ec` outputs [3]
Frozen authored proofs	82 manifested EasyCrypt targets under `haetae-topdown-easycrypt/easycrypt/`; both the pre-freeze 82/82 and current frozen-suite 82/82 results are preserved
Frozen paper decision	`PAPER-FROZEN`; internal stopping points are `STOP-KG-NTT`, `STOP-SIGN-CHAL-MODE2`, `STOP-VERIFY-MATRIX-CRT`
Latest post-freeze decision	`STOP-KG-AVEC-QJ`; `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS` is the first repair obligation. Pure NTT/Rq and actual-a/paper-qj auxiliary theorems are verified separately outside the frozen manifest
Reused proofs	Import-only adapters retaining the original premises of pinned KeyGen, NTT, sampler, singularity, and FFT results [1]
Excluded scope	Other parameter sets, physical side channels, faults, RNG quality, and memory safety and SCT that do not follow automatically from EUF-CMA

What this guide does not claim

The work does not yet claim complete KeyGen distribution equality, full Sign/Verify functional correctness, equality of public-context transcripts, direct μ equality for accepted raw-API traces, the complete 1474-byte signature codec, actual rANS success reachability for general canonical inputs, encoder/decoder termination, unconditional round trips of the actual complete HBZ wrappers on every canonical input, the (h) codec, identification with full-NTT/paper multiplication within the frozen claim matrix, identification of the security model's ExpandVecA with actual `avec`, the paper's KeyGen equation, MINCORE Sign/Verify composition, encoding zero loss, or end-to-end EUF-CMA security of the Jasmin implementation. The success postcondition for fixed all-zero input is proved, but termination/losslessness of that execution is separate.

2 Mathematical targets at the paper level

2.1 Minimal HAETAE-2 background

The paper describes HAETAE as a module-lattice signature scheme over

$$R = \mathbb{Z}[X]/(X^{256} + 1), \qquad R_q = \mathbb{Z}_q[X]/(X^{256} + 1), \quad q = 64513$$

[2]. In mode 2, $(k, \ell) = (2, 4)$, the small-secret coefficient bound is $\eta = 1$, and the binary challenge has weight $\tau = 58$. The verification key is compressed as $vk = (seedA, b_1)$; its mode-2 length in the Jasmin API is 992 bytes.

The core data flow of Figure 8 in the paper, restricted to what this guide needs, is as follows.

$$
\begin{aligned}
b &= a_{\text{gen}} + A_{\text{gen}} s_{\text{gen}} + e_{\text{gen}}, & (b_0, b_1) &= (\text{LowBits}_{vk}(b), \text{HighBits}_{vk}(b)),\\
vk &= (seedA, b_1), & \mu &= H_{\text{gen}}(seedA, b_1, M),\\
\rho &= H(\text{HighBits}_h(w), \text{LSB}(\lfloor y_0 \rceil), \mu), & c &= \text{SampleBinaryChallenge}_\tau(\rho).
\end{aligned}
$$

Sign constructs μ from the verification-key component stored inside the secret key, while Verify reconstructs it from the public verification key. Both then place highbits, LSB, and μ in the challenge transcript. The first implementation-correctness composition point therefore consists of three concrete equalities, rather than merely saying that "SHAKE is correct."

1. The packed secret-key prefix read by Sign equals the packed verification-key bytes read by Verify.
2. After absorbing the same pre and message, the first 32 bytes of Sign's 64-byte μ equal Verify's 32-byte μ.
3. Absorbing those 32 bytes at position 64 of the same challenge state produces equal final states and positions.

The work connected these three equalities to actual generated procedures on raw/internal paths, tracing the control/memory facts by which actual raw-API callers reach the inputs, the Sign output frame, and hash equivalence requiring only the regions read. Directly connecting the two μ values stored in completed traces was frozen at a product/replay semantic boundary. The main execution path in Weeks 8–15 was the actual signature codec's HBZ rANS suffix. Coefficient/symbol leaves, tables, pure single steps, the common byte stack, actual suffix copying, and 3-call harness control were closed. Week 10 added actual encoder inner-byte writes and successful size bounds. Week 11 composed exact 0/1/2-byte progress preserving the pure tail in the inner invariant, generated table word updates, the outer one-symbol transition, and the final four stores, identifying the complete actual successful encoder suffix with the pure trace. Week 12 closed exact consumption of that trace by the actual decoder, and Week 13 closed the actual encoder–suffix-copy–decoder harness composition. Week 14 composed this core inverse with actual prepare/apply and production full-HBZ wrapper boundaries. Week 15 proved nonzero-size success for terminating actual encoder executions on the all-6 symbol sequence produced by canonical all-zero HBZ. Week 16 compiled the KeyGen matrix/finalizer snapshot, the final sound NTT representation rewrite of `output_row`, and the actual harness's two row consequences as target 77. The required convolution layer was absent from the frozen surface, so the boundary was fixed at `STOP-KG-NTT`; limited Sign/Verify results then completed the 82-target `PAPER-FROZEN` boundary. Post-freeze work closed pure NTT/Rq representations and the actual-(a)/paper-(qj) auxiliary layer, but the synthetic security object's semantic mismatch makes `STOP-KG-AVEC-QJ` the latest gate.

2.2 Functional-correctness target for the mode-2 signature codec

An actual mode-2 signature has 1474 bytes. The extracted packer/unpacker layout is as follows.

$$
\underbrace{[0, 32)}_{\text{256 challenge bits}} \;\|\; \underbrace{[32, 1056)}_{\text{1024 signed low bytes}} \;\|\; \underbrace{[1056, 1058)}_{\text{two size deltas}} \;\|\; \underbrace{[1058, 1474)}_{\text{hbz/h payload and canonical zero padding}}.
$$

The final region has a total budget of 416 bytes, giving $1056 + 2 + 416 = 1474$. For a canonical challenge c and a low-part array z_{low} in the signed-byte range, the closed prefix target is

$$\mathrm{UnpackPrefix}(\mathrm{PackPrefix}(c, z_{\mathrm{low}})) = (c, z_{\mathrm{low}})$$

This equality is proved as partial correctness for actual generated procedures. It excludes the two rANS payloads, size metadata, zero padding, and success and uniqueness of the complete parse. The Week 8–12 results further divide the internal layers of the first, HBZ payload.

2.3 Mathematical structure of HBZ leaves, rANS single steps, and byte traces

A mode-2 HBZ input consists of 1024 coefficients $h_i \in [-6, 7)$. The actual prepare procedure maps them to

$$s_i = h_i + 6 \in \{0, \dots, 12\}$$

The actual apply procedure reconstructs $s_i - 6$ as a 32-bit word. Thus terminating composition of the two actual leaves on a canonical HBZ array satisfies

$$\mathrm{Apply}(\mathrm{Prepare}(h))[0..1024) = h[0..1024)$$

The prepare theorem separately preserves the symbol tail of capacity 2048, and the composition postcondition preserves the decoded-coefficient tail. This is leaf-level partial correctness, with no claim about rANS buffer success or size.

The cumulative starts and frequencies of the 13 symbols are respectively

$$(c_s)_{s=0}^{12} = (0, 1, 2, 3, 8, 66, 312, 710, 957, 1016, 1021, 1022, 1023),$$
$$(f_s)_{s=0}^{12} = (1, 1, 1, 5, 58, 246, 398, 247, 59, 5, 1, 1, 1),$$

Here $f_s > 0$, $\sum_s f_s = 1024$, and the intervals $[c_s, c_s + f_s)$ partition $[0, 1024)$. Define the pure rANS steps by

$$E_s(x) = \left\lfloor \frac{x}{f_s} \right\rfloor 1024 + c_s + (x \bmod f_s),$$
$$D_s(y) = \left\lfloor \frac{y}{1024} \right\rfloor f_s + (y \bmod 1024) - c_s$$

The current proof establishes $D_s(E_s(x)) = x$ for $x \geq 0$. It also proves that the fast step computed by the actual mode-2 reciprocal/bias expression equals $E_s(x)$ when $0 \leq s < 13$ and $1 \leq x < \mathsf{hbz_xmax}(s)$. The literal `esyms/dsyms_words/symbol_words` arrays in the actual packing/unpacking extractions satisfy this certificate.

Week 9 fixed the pure byte trace linking single steps as follows. Let the symbols be $s_0, \dots, s_{n-1}$, with $n = 1024$, and set $x_n = L = 2^{23}$. In encoder order $j = n - 1, \dots, 0$,

$$\ell_j = \mathrm{renorm_len}(x_{j+1}, \mathsf{hbz_xmax}(s_j)) \in \{0, 1, 2\},$$
$$r_j = \left\lfloor x_{j+1} / 256^{\ell_j} \right\rfloor,$$
$$x_j = E_{s_j}(r_j).$$

Let β_j be the normalization-byte segment in the order read by the actual decoder. The compiled readback theorem gives

$$\mathrm{ReadBytes}(r_j, \beta_j) = x_{j+1}$$

In particular, when emitting two bytes, the actual encoder writes the low byte first while decrementing its cursor, so the final forward segment is $[\mathsf{high}, \mathsf{low}]$. The common trace is therefore

$$c_0 = 4, \qquad c_{j+1} = c_j + |\beta_j|, \qquad B = \mathrm{LE32}(x_0) \| \beta_0 \| \cdots \| \beta_{n-1}$$

Here ParseLE32 restores the first state; each D_{s_j} obtains r_j from x_j, then reads β_j to recover x_{j+1}. The pure induction theorem preserves

$$2^{23} \le x_j < 2^{31}$$

at every boundary state.

These pure results alone do not imply refinement of the actual encoder and decoder outer loops. Compiled results also include the Hoare theorem that actual `__copy_encoded_suffix` copies the encoder suffix pointwise to a decoder prefix while preserving the remaining tail, and the unary harness-control theorem that performs or skips copying and decoding according to the actual encoder's returned *bad*.

2.4 Week 10 mathematical boundary of the actual encoder

For a symbol array a of capacity 2048, define the list used by mode 2 and its suffix by

$$S(a) = [\mathrm{uint}(a[0]), \dots, \mathrm{uint}(a[1023])], \qquad S_i(a) = \mathrm{drop}_i(S(a))$$

These are exactly `symbol_list_of_array` and `symbol_suffix` in the code. Read the array segment and frame conditions as

$$\begin{aligned} \mathrm{Segment}(A,p,b) &\iff 0 \le p \ \wedge \ p + |b| \le 2048 \ \wedge \\ &\qquad \forall k \in [0,|b|),\ \mathrm{uint}(A[p+k]) = b[k], \\ \mathrm{PrefixFrame}(A_0,A_1,p) &\iff \forall k \in [0,p),\ A_0[k] = A_1[k] \end{aligned}$$

The compiled array/list bridges give $S_{1024}(a) = []$ and $S_i(a) = \mathrm{uint}(a[i]) :: S_{i+1}(a)$. They also isolate the one-symbol trace recurrence as follows.

$$\begin{aligned} T(s :: t).\mathsf{state} &= E_s(\mathrm{Norm}(T(t).\mathsf{state}, s)), \\ T(s :: t).\mathsf{bytes} &= \mathrm{NormBytes}(T(t).\mathsf{state}, s) \| T(t).\mathsf{bytes}. \end{aligned}$$

The word step modeling the actual table's four words, 64-bit reciprocal multiplication and high-half extraction, shift ladder, and bias satisfies

$$\begin{gathered} 0 \le s < 13, \qquad 1 \le \mathrm{uint}(x) < \mathsf{hbz_xmax}(s), \\ \mathrm{ActualWordStep}(x,s) = \mathrm{W32}(\mathrm{FastStep}(\mathrm{uint}(x), s)) \end{gathered}$$

Combined with the preceding fast-step theorem, the right side is the W32 representation of mathematical E_s. This is a word-level equality, not a Hoare theorem for the generated outer loop.

The generated `_rans_encode` inner normalization loop carries the following local invariant, with witnesses for entry state (x_0, off_0), threshold $x_{\max}$, and step k.

$$\begin{gathered} 4 \le off_0 \le 1024, \qquad 0 \le k \le 4, \qquad 1 \le \mathrm{uint}(x_{\max}), \\ off = off_0 - k, \qquad x = \mathrm{PhaseState}(x_0, k), \\ \mathrm{Segment}(encp, off_0 - k, \mathrm{PhaseBytes}(x_0, k)), \qquad \mathrm{PrefixFrame}(before, encp, off_0 - k). \end{gathered}$$

It directly proves that each actual `set8` step prepends the low byte to the list, the W64 cursor does not underflow, and the unwritten prefix is preserved. However, *before* is the array at entry to that inner loop, so the invariant does not yet describe the complete accumulated outer trace. The bound $0 \le k \le 4$ is conservative, derived from W32 finiteness; identification with the pure trace's exact 0/1/2-byte length remains open.

Consequently, two partial-correctness theorems for the actual generated procedure establish

$$\begin{aligned} bad' \neq 0 \quad &\vee \quad (bad' = 0 \wedge 0 \leq off' \leq 1020), \\ bad' = 0 \quad &\Longrightarrow \quad 4 \leq 1024 - off' \leq 1024 \end{aligned}$$

The second formula bounds encoded size on success. The conclusions do not include reachability or termination of a successful path, or success on input whose symbols are all 6 (all-6 input).

2.5 Week 11 closure of the actual encoder trace

Week 11's tail-aware inner invariant carries the already written pure tail t, in addition to the local bytes.

$$\begin{gathered} 0 \leq k \leq \ell(x_0, s), \qquad off = off_0 - k, \\ \mathrm{Segment}(encp, off_0 - k, \mathrm{Written}(x_0, s, k) \| T(t).\mathsf{bytes}), \qquad \mathrm{PrefixFrame}(enc_0, encp, off_0 - k). \end{gathered}$$

Here $\ell(x_0, s) \in \{0, 1, 2\}$ is the pure normalization length. Proving $k = \ell(x_0, s)$ when the actual generated guard becomes false closes the gap between Week 10's conservative 0..4 phase and the pure trace.

The successful-path outer invariant is

$$\begin{gathered} 0 \leq i \leq 1024, \qquad x = T(S_i(a_0)).\mathsf{state}, \qquad 4 \leq off, \\ off + |T(S_i(a_0)).\mathsf{bytes}| = 1024, \\ \mathrm{Segment}(encp, off, T(S_i(a_0)).\mathsf{bytes}), \qquad \mathrm{PrefixFrame}(enc_0, encp, off) \end{gathered}$$

Dedicated word theorems fold generated table reads and protection operations, reciprocal multiplication, and the shift ladder to recover the one-symbol transition. At $i = 0$, the four actual little-endian stores are composed. A terminating actual `_rans_encode` thus either returns failure or satisfies the following postcondition on success.

$$\begin{aligned} bad' \neq 0 \quad \vee \quad (bad' = 0 \wedge\ & 0 \leq off' \leq 1020 \wedge 4 \leq 1024 - off' \leq 1024 \\ & \wedge \mathrm{Segment}(encp', off', \mathrm{trace_bytes}(S(a_0))) \\ & \wedge off' + |\,\mathrm{trace_bytes}(S(a_0))| = 1024 \\ & \wedge \mathrm{PrefixFrame}(enc_0, encp', off')). \end{aligned}$$

`actual_rans_encode_trace_closure` and `actual_rans_encode_trace_refinement` prove this statement for the actual generated procedure. Thus **OBL-RANS-ENCODE-REFINEMENT** is **PROVED**as success-conditioned partial correctness. Success is not a precondition, but the theorem does not prove termination or reachability of $bad' = 0$.

2.6 Week 12 closure of the actual decoder trace

For the expected symbol array a of capacity 2048, define

$$s := S(a), \qquad B := \mathrm{trace_bytes}(s), \qquad n := |B|$$

The actual decoder's exact-trace input requires `mode2_hbz_symbol_stream`(a), $4 \leq n \leq 1024$, pointwise agreement of the input-buffer segment $[0, n)$ with B, state fields

$$(\mathsf{count}, \mathsf{size}, m) = (1024, n, 13)$$

and bindings for the actual `symbol_words/dsyms_words` tables. It also explicitly requires pointwise reads within each normalization segment to equal the corresponding pure list elements. This input predicate contains no decoder success, output-symbol equality, or post-state assumption.

Defining the reference state, cursor, and segment by

$$x_i := \text{decoder_state_at}(s, i), \qquad c_i := \text{decoder_cursor}(s, i),$$
$$\beta_i := \text{decoder_segment_at}(s, i)$$

the compiled boundary theorem gives

$$c_0 = 4, \qquad c_{i+1} = c_i + |\beta_i|, \qquad c_{1024} = n, \qquad x_{1024} = 2^{23}$$

The actual inner loop carries $k = \mathsf{off} - c_i$ and the partially replayed state, consuming precisely the next byte or stopping according to segment length $|\beta_i| \in \{0, 1, 2\}$. Separate theorems connect concrete packed-table lookup and W32/W64 operations to selection of the same symbol and mathematical decoder step.

Consequently, `actual_rans_decode_trace_refinement` proves the following for every terminating execution of the actual generated `RansDecodeTarget.M._rans_decode`:

$$bad_{\text{dec}} = 0, \qquad off_{\text{dec}} = n,$$
$$decoded'[0..1024) = a[0..1024), \quad decoded'[1024..2048) = decoded_0[1024..2048)$$

The internal fact $x_{1024} = 2^{23}$ excludes the actual final rejection check, but is not exposed in the generated ABI result. Thus **OBL-RANS-DECODE-REFINEMENT** is **PROVED**as exact-trace partial correctness. Termination, losslessness, and actual encoder-success reachability are not conclusions of this theorem.

2.7 Week 13 closure of actual rANS core composition

Week 13 sequentially composed the preceding three individual theorems inside actual `Mode2RansActualHarness . run`. This harness directly calls the fixed extractions' `_rans _ encode`, `_ _ copy _ encoded _ suffix`, and `_rans _ decode`, in that order. `Mode2RansCoreCompositionBridge.ec` constructs the copied buffer's exact trace from the encoder's `segment _matches` and actual copying's `slice_eq`, then transports it to every pointwise decoder read and the constructed state fields $(1024, n, 13)$. It also derives from the actual successful size range that W64 subtraction equals the integer trace size without wraparound.

The only preconditions of `actual_rans_encode_copy_decode_inverse` are initial argument bindings and `mode2_hbz_symbol_stream`. In particular, encoder success and copy/decoder results are not preconditions. For every terminating execution, the conclusion is approximately

$$(bad_{\text{enc}} \neq 0 \wedge \neg\mathsf{decoder_ran} \wedge decoded' = decoded_0)$$
$$\vee\ (bad_{\text{enc}} = 0 \wedge \mathsf{decoder_ran} \wedge bad_{\text{dec}} = 0 \wedge off_{\text{dec}} = \mathsf{size}$$
$$\wedge\ decoded[0..1024) = symbols[0..1024) \wedge \mathsf{frames})$$

On failure the decoder state also retains its initial value. On success, the decoder input fields, actual encoder trace suffix, and initial-prefix frame are preserved as well. Thus **OBL-RANS-CORE-INVERSE** is **PROVED**as success-conditioned Hoare partial correctness.

This is a theorem about a proof-authored harness directly calling three actual generated procedures, not yet a wrapper theorem for production `_encode_hb_z1_full/_decode_hb_`

`z1_full`. Canonical h alone also does not imply buffer success, so the full HBZ wrapper target must still expose at least the actual failure branch as follows:

$$\mathsf{size} = 0 \quad \vee \quad (\mathsf{size} \neq 0 \wedge \mathsf{bad} = 0 \wedge \mathsf{decoded}[0..1024) = h[0..1024))$$

Transport of the full HBZ parent's frame and size, encoder/decoder termination, and an actual success witness for all-6 input had not yet been compiled.

2.8 Week 14 closure of production full-HBZ wrappers

Week 14 connected the focused full-HBZ path calling `_encode_hb_z1_full` followed by `_decode_hb_z1_full` to the production SignaturePack/Unpack path, showing that they expose the same actual procedure semantics. `Mode2HbzInternalBoundaries.ec` establishes exact equivalence between focused/full extractions of prepare, rANS encoder, rANS decoder, and apply. It provides adapters from canonical coefficients to a 13-symbol stream and from recovered symbols back to apply premises.

The only semantic premises of `actual_hbz_full_encode_decode_inverse_mode2` are initial argument bindings and `canonical_hbz_mode2`. Transported to the production boundary, `signature_pack_unpack_hbz_full_inverse_mode2` proves one of the following two branches for every terminating execution.

$$\begin{aligned}
&\mathsf{size} = 0 \Longrightarrow \neg\mathsf{decoder_ran} \wedge \mathit{encoded}' = \mathit{encoded}_0 \wedge \mathit{decoded}' = \mathit{decoded}_0 \wedge \mathit{bad}' = \mathit{bad}_0,\\
&\mathsf{size} \neq 0 \Longrightarrow 4 \leq \mathsf{size} \leq 1024 \wedge \mathsf{decoder_ran} \wedge \mathit{bad}' = 0\\
&\qquad\qquad \wedge\ \mathit{decoded}'[0..1024) = \mathit{hbz}_0[0..1024) \wedge \mathsf{coefficient/encoded\ frames}.
\end{aligned}$$

The success branch also includes the `prepared_symbols` witness produced by actual prepare and equality of its `mode2_trace_bytes` with the returned encoded segment. `signature_pack_unpack_hbz_full_actual_exact` first proves exact equality of the production and focused harnesses' result tuples and `decoder_ran`, so generated-procedure semantics are not assumed as a new axiom.

The following obligation is therefore closed as success-conditioned Hoare partial correctness.

OBL-SIG-HBZ-ENCODE-DECODE: **PROVED**

However, this statement does not eliminate the $\mathsf{size} = 0$ branch. An actual success witness for all-zero coefficients (all-6 symbols after prepare) and encoder/decoder termination remain separate obligations. Before those are closed, unconditional round trips or $\delta_{\text{Encoding}} = 0$ cannot be concluded.

2.9 Week 15 actual success postcondition for fixed all-6 input

Week 15 closed the previously isolated fixed-input edge with two authored theories. `Mode2RansAllSixBudget.ec` unfolds every 32-bit coefficient read of bytewise-zero-initialized `zero_hbz` to 0, establishing canonicality, and proves that the first 1024 symbols produced by actual prepare are all 6.

The frequency and threshold for symbol 6 are

$$\mathsf{freq}(6) = 398, \qquad \mathsf{xmax}(6) = 2^{21} \cdot 398 = 834666496$$

When the state is below 2^{31}, one symbol emits at most one normalization byte, and the first four pure states

$$8388608 \to 21582496 \to 55528910 \to 142868116 \to 367580518$$

emit no bytes. Thus the normalization portion of 1024 all-6 symbols uses at most 1020 bytes. Including 4 bytes for final-state serialization, the complete trace satisfies

$$4 \le |\mathsf{mode2_trace_bytes}| \le 1024$$

The strengthened actual encoder-closure theorem preserves

$$bad \neq 0 \Longrightarrow 1020 < |\mathsf{normalization\ trace}|$$

This contradicts the pure bound above. Hence `actual_rans_encode_all_six_success` concludes $bad = 0$, exact trace and size, and the prefix frame for terminating actual `_rans_encode` executions. Lifting this to full-HBZ and production boundaries, `signature_pack_unpack_hbz_zero_success_mode2` proves, on fixed `zero_hbz` input,

$$size \neq 0, \quad 4 \le size \le 1024, \quad \mathsf{decoder_ran}, \quad bad_{\mathrm{dec}} = 0, \quad decoded[0..1024) = 0$$

and both tail frames.

This makes **OBL-RANS-ACTUAL-SUCCESS-WITNESS** **PROVED**as Hoare partial correctness for fixed all-6 input. A hoare statement, however, does not establish that the execution exists or terminates. Fixed-input losslessness, phoare $= 1$, success on general canonical inputs, and encoder/decoder termination remain separate obligations.

2.10 Week 16 actual KeyGen matrix/finalizer snapshot

The actual boundary of the paper's KeyGen equation was restricted to two fixed helpers, rather than opening the entire retry/API layer at once. `ActualM23MatrixFinalizeSnapshot.run` calls actual `_kp_m23_matrix` with mode-2 dimensions $(2, 3)$, stores its returned array as $p = \mathsf{pre_bp}$, then calls actual `_keypair_finalize_m23` on 512 words.

For each active coefficient, let

$$r = (p_i + s_{2,i} + a_i) \bmod q, \qquad b_{0,i} = \mathrm{Low}(r), \qquad s'_{2,i} = s_{2,i} - b_{0,i}$$

The compiled semantic theorem then gives

$$r = 2b_{1,i} + b_{0,i}, \qquad -1 \le b_{0,i} \le 1$$

and the snapshot-scoped congruence

$$2(a_i - 2b_{1,i}) + 2p_i + 2s'_{2,i} \equiv 0 \pmod{2q}$$

These are the actual finalizer's KG-2-style low/high decomposition and the adjusted s_2 component of KG-4. The matrix/NTT representation and all three array-tail frames are preserved as well.

Crucially, p is the `KeygenM23ArithmeticSpec.output_row` snapshot returned by the actual matrix helper. `output_row_from_mode2_ntt_words` compiles the final representation edge expanding this value as

$$\mathrm{array256_mont}(\mathrm{full_invntt}(\mathrm{pointwise_row_words}(m, v, row)))$$

`actual_m23_matrix_snapshot_rows_explicit` transports it to the two active-row postconditions of the actual two-call harness. This expression is not renamed to the paper's $A_{\mathrm{gen}} s_{\mathrm{gen}}$. At paper freeze, the first missing leaf identified by the audit was odd-root orthogonality/full-NTT spectral action with the actual Montgomery factor: identifying inverse NTT of the pointwise product with `Rq.&*` negacyclic convolution. A separate adapter

was then required from `Rq.poly` arrays to `HAETAE_Algebra.poly_mul` integer lists. Hence, in the frozen claim matrix, faithful KG-1, KG-3, complete KG-4, and

$$As = qj \pmod{2q}$$

were not proved. The frozen `STOP-KG-NTT` decision fixed KeyGen at KG-2 and the finalization boundary. Direct calls to the three actual Sign helpers and accepted-branch control were subsequently compiled, but the following theorem is absent.

`sf_challenge_mode2_highbits_lsb_sampleinball_correct`

The boundary was therefore frozen as `STOP-SIGN-CHAL-MODE2`. The S-1–S-7 headline theorem is unproved. The subsequent MINCORE-VERIFY work preserved helper-local machine-word and control results.

After paper freeze, `rq_mul_coeff_foldr_to_bigi`, `full_ntt_montgomery_spectral_action`, general row products, and the Rq–HAETAE multiplication-representation adapter were closed in a separate auxiliary tree. Actual mode-2 `avec` was also connected to the paper's a through its SHAKE128/rejection semantics, and the paper's qj term was added independently to the snapshot equation. However, the existing security model's synthetic `haetae_mode23_qj_vector` was proved different from these objects, yielding the latest decision `STOP-KG-AVEC-QJ`. KG-1–KG-4 and the paper equation above therefore remain unpromoted. For Verify, the first procedure blockers of the combined theorem are actual 2-by-4 `verify_matrix_ntt_acc_mode2_cols4_correct` and `verify_crt_freeze_mode2_word_exact`, rather than the pure NTT foundation.

2.11 Top-level functional-correctness targets

`easycrypt/specs/TopLevelGoals.ec` models the implementation and the paper's probabilistic algorithms as abstract operations and defines the following targets.

$$\begin{aligned}
\mathsf{FC}_{\mathrm{KG}}: &\quad \mathsf{JKeyGen}_2 = \mathsf{PaperKeyGen}_2,\\
\mathsf{FC}_{\mathrm{Sign}}: &\quad \forall sk, M,\ \mathsf{JSign}_2(sk, M) = \mathsf{PaperSign}_2(sk, M),\\
\mathsf{FC}_{\mathrm{Verify}}: &\quad \forall vk, M, \sigma,\ \mathsf{JVerify}_2(vk, M, \sigma) = \mathsf{PaperVerify}_2(vk, M, \sigma).
\end{aligned}$$

These formulas are specification interfaces to be implemented, not declarations of completed proofs. The current prefix and transcript theorems are deterministic subtheorems required by these equalities.

2.12 Top-level security-composition target

The target comparing the implementation EUF-CMA advantage with that of the paper scheme is

$$\mathrm{Adv}^{\text{euf-cma}}_{\text{Jasmin}} \le \mathrm{Adv}^{\text{euf-cma}}_{\text{paper}} + \delta_{\text{KeyGen}} + q_{\text{Sign}}\delta_{\text{Sign}} + \delta_{\text{Verify}} + \delta_{\text{Encoding}} \tag{1}$$

At the paper level, the further target is

$$\mathrm{Adv}^{\text{euf-cma}}_{\text{paper}} \le \mathrm{Adv}_{\text{MLWE}} + \mathrm{Adv}_{\text{BST-MSIS}} + \varepsilon_{\text{ROM}} + \varepsilon_{\text{Rejection}} + \varepsilon_{\text{Fork}} \tag{2}$$

Equations equations (1) and (2) also remain **SPECIFIED**or conditional connections. The final cryptographic-assumption interfaces may be limited to HAETAE-2 MLWE, BST-MSIS, and ROM, but the concrete game instances and reduction theorems are not all connected yet.

3 The formal language for reading statements: EasyCrypt

3.1 Core predicates and relations

Definition 3.1 (Packed-key prefix)**.**

$$\mathrm{VKPrefix}_n(sk, vk) \iff \forall i\ (0 \le i < n \Rightarrow sk[i] = vk[i]).$$

This is `vk_prefix_eq` in the code; mode 2 uses $n = 992$.

Definition 3.2 (Prefix agreement of external key regions)**.**

$$\mathrm{ExternalKeyPrefix}(mem, sku, vku) \iff \forall i \in [0, 992),\ \mathrm{load8}(mem, sku{+}i) = \mathrm{load8}(mem, vku{+}i).$$

This is `external_key_prefix_match`. It compares the first 992 bytes exported by actual raw KeyGen to distinct external SK/VK addresses.

Definition 3.3 (Prefix in memory)**.**

$$\mathrm{SKMemPrefix}(sk, mem, b) \iff \forall i \in [0, 992),\ sk[i] = \mathrm{load8}(mem, \mathrm{uint}(b) + i).$$

This predicate compares Sign's local `BArray2752` with Verify's raw-memory reads in one relation.

Definition 3.4 (Canonical address range)**.** For an integer address a and length n,

$$\mathrm{CanonicalAddress}(a) \iff 0 \le a < 2^{64},$$
$$\mathrm{CanonicalRegion}(a, n) \iff 0 \le a \ \wedge\ 0 \le n \ \wedge\ a + n \le 2^{64}.$$

The first premise permits a raw application binary interface (raw ABI) integer address to round-trip through `W64.of_int` and `W64.to_uint`; the second excludes modular wraparound of the address range. For length 0, the second formula alone also permits $a = 2^{64}$, so the actual round-trip theorems separately require `canonical_ui64_address` for the necessary addresses and lengths. This is a call contract, not a proof of physical memory safety.

Definition 3.5 (Memory-region equality)**.**

$$\mathrm{RegionEq}(m_1, m_2, b, n) \iff \forall i \in [0, \mathrm{uint}(n)),\ \mathrm{load8}(m_1, \mathrm{uint}(b) + i) = \mathrm{load8}(m_2, \mathrm{uint}(b) + i).$$

This is `region_eq` in the code. Week 6 hash theorems require only this relation on the regions actually read in VK, pre, and message, rather than whole-memory equality.

Definition 3.6 (Canonical signature-prefix inputs)**.** `canonical_challenge` means that each of the 256 challenge words is exactly 0 or 1. `canonical_signed_low` means that each of the 1024 low words equals the sign extension of its low byte, with signed integer value in $[-128, 128)$. The actual prefix round trip holds under these two premises.

Definition 3.7 (Canonical mode-2 HBZ and leaf frames)**.** For a 32-bit coefficient array h of capacity 2048,

$$\mathrm{CanonicalHBZ}_2(h) \iff \forall i \in [0, 1024),\ -6 \le \mathrm{sint}(h[i]) < 7.$$

This is `canonical_hbz_mode2` in the code. `prepared_hbz_prefix` says that the first n symbol bytes equal $\mathrm{sint}(h[i]) + 6$; `decoded_hbz_prefix` says that the first n recovered coefficients equal the originals. `byte_tail_frame` and `coeff_tail_frame` record that $[1024, 2048)$ is unchanged before and after leaf execution in their respective arrays.

Definition 3.8 (Mode-2 HBZ table certificate)**.** `mode2_hbz_table_certificate` is the conjunction of three facts.

1. The four fields of the actual encoder table for each of the 13 symbols equal their specifications.
2. Actual decoder words contain each interval's start/frequency exactly.
3. The actual 512 packed symbol words map each of 1024 slots to its unique interval symbol.

`actual_mode2_hbz_tables_certified` applies this predicate to the literal arrays in the pack/unpack extractions. It does not assume correctness of the external certificate-generation script: the kernel rechecks every generated EasyCrypt equality.

Definition 3.9 (Canonical rANS byte trace)**.** For a canonical symbol list $s = (s_0, \ldots, s_{1023})$, `encode_trace` processes symbols in reverse from $x_{1024} = 2^{23}$, determining boundary states $(x_j)_{j=0}^{1024}$ and decoder-order byte segments $(\beta_j)_{j=0}^{1023}$. The cursor cuts and complete byte sequence are

$$c_0 = 4, \qquad c_{j+1} = c_j + |\beta_j|, \qquad B = \text{serialize32_le}(x_0) \| \beta_0 \| \cdots \| \beta_{1023}$$

The code's `valid_rans_trace`(s,xs,cuts,bytes) is simply the conjunction equating its arguments to these three objects computed by `trace_states`, `trace_cuts`, and `trace_bytes`. It assumes neither existence of an external trace nor decoder success; `valid_rans_trace_canonical` proves the canonical construction.

Definition 3.10 (Array/list bridge and segment frame)**.** For a byte array a of capacity 2048, `symbol_list_of_array`(a) is the integer list of its first 1024 elements, and `symbol_suffix`(a,i) drops the first i elements from that list. `segment_matches`(a,p,bs) means

$$0 \le p \ \wedge \ p + |bs| \le 2048 \ \wedge \ \forall k \in [0, |bs|), \quad \text{uint}(a[p+k]) = bs[k]$$

The predicate `prefix_frame`(before,after,p) says that the arrays' k-th bytes agree for every $k < p$. Week 10's compiled theorems provide

$$\begin{aligned} &\mathsf{symbol_suffix}(a, 1024) = [\,], \\ &\quad \mathsf{symbol_suffix}(a, i) = \text{uint}(a[i]) :: \mathsf{symbol_suffix}(a, i+1) \end{aligned}$$

together with rules for segment concatenation, prepending a one-byte write, and shrinking frames.

Definition 3.11 (Actual encoder inner-step invariant)**.** `encoder_inner_phase_inv` and `encoder_inner_segment_inv` directly open the generated `_rans_encode` inner normalization loop. They assert existence of entry state (x_0, off_0) and step k such that

$$\begin{aligned} 4 \le off_0 \le 1024, \qquad 0 \le k \le 4, \qquad 1 \le \text{uint}(x_{\max}), \\ \text{uint}(off) = off_0 - k, \qquad \text{uint}(x) = \mathsf{encoder_phase_state}(x_0, k), \\ \mathsf{segment_matches}(encp, off_0 - k, \mathsf{encoder_phase_written}(x_0, k)) \end{aligned}$$

and preservation of the lower prefix relative to the array at entry to that inner call. These are Week 10's conservative local predicates, verifying the actual one-byte store and W64 cursor safety. Week 11's stronger tail-aware predicate adds outer accumulation and the exact 0/1/2-byte length.

Definition 3.12 (Tail-aware inner invariant)**.** Week 11's `encoder_inner_tail_inv` combines the exact pure normalization length $\ell = \mathsf{mode2_normalization_len}(x_0, s)$ with the encoding trace of the existing pure symbol tail *tail*. Its core segment condition is

$$\mathsf{segment_matches}(encp, off_0 - k, \mathsf{inner_written_suffix}(x_0, s, k) \mathbin{+\!\!+} \mathsf{encode_trace}(tail).\mathsf{bytes})$$

along with $0 \le k \le \ell$, cursor and state equalities, and the prefix frame. The predicate is preserved through actual-guard progress and exit, yielding exactly $k = \ell \le 2$. It is then used in the outer one-symbol recurrence and final-store composition to close the actual outer procedure's complete successful postcondition.

Verified theorem 3.1 (Success-conditioned complete suffix of the actual encoder)**.** `actual_rans_encode_trace_closure` directly opens the actual generated `_rans_encode`, proving either returned $bad \neq 0$ or, when $bad = 0$,

$$\mathsf{segment_matches}(encp', \mathrm{uint}(off'), \mathsf{trace_bytes}(\mathsf{symbol_list_of_array}(symbols_0)))$$

together with $off' + |\mathsf{trace_bytes}| = 1024$, the successful size range, and `prefix_frame`. Success is not a precondition. Because this is Hoare partial correctness, termination and success reachability remain separate obligations.

Definition 3.13 (Actual decoder exact-trace input)**.** `actual_mode2_decoder_trace_input(a,b,st,n)` is a functional representation contract between expected symbol array a, encoded buffer b, initial state st, and logical size n. Its core conditions are

$$\mathsf{mode2_hbz_symbol_stream}(a), \qquad n = |\mathsf{trace_bytes}(\mathsf{symbol_list_of_array}(a))|,$$
$$4 \le n \le 1024, \qquad \mathsf{segment_matches}(b, 0, \mathsf{trace_bytes}(\cdots)),$$
$$(st[0], st[1], st[2]) = (1024, n, 13)$$

and pointwise read equalities for every normalization segment. The actual mode-2 `symbol_words` and `dsyms_words` arrays are fixed by separate bindings in the top-level Hoare theorem. This predicate does not assume $bad = 0$, an output-array equality, or a post-state.

Definition 3.14 (Decoder outer/inner trace invariants)**.** At iteration i, `decoder_outer_trace_live` binds the actual state and cursor to

$$x = \mathsf{decoder_state_at}(s, i), \qquad \mathrm{uint}(off) = \mathsf{decoder_cursor}(s, i)$$

while maintaining the expected symbols in $decoded[0..i)$ and the input-array frame in $decoded[i..2048)$. After a one-symbol word update, `decoder_inner_trace_live` uses

$$k = \mathrm{uint}(off) - \mathsf{decoder_cursor}(s, i),$$
$$x = \mathsf{decoder_replay_prefix}(\mathsf{decoder_state_at}(s, i+1), s_i, k)$$

and maintains $0 \le k \le |\mathsf{decoder_segment_at}(s, i)|$. The generated guard agrees exactly with $k < |\mathsf{segment}|$, and the segment length is 0, 1, or 2 bytes. Thus each actual read is either a one-byte consumption transition or a no-consumption exit to the next outer boundary.

Verified theorem 3.2 (Actual decoder exact-trace recovery)**.** `actual_rans_decode_trace_refinement` directly opens actual generated `RansDecodeTarget.M._rans_decode` under definition 3.13 and bindings for the two actual tables. Every terminating execution satisfies

$$bad' = 0, \qquad off' = n, \qquad decoded'[0..1024) = a[0..1024)$$

and the output tail $[1024, 2048)$ equals the initial array. The internal $x = 2^{23}$ at outer-loop exit excludes the actual final rejection check, but is absent from the ABI postcondition. This is exact-trace Hoare partial correctness, not a proof of termination or actual encoder success.

Definition 3.15 (Actual rANS harness control)**.** `Mode2RansActualHarness.run` calls actual `_rans_encode`, then calls actual `__copy_encoded_suffix` and `_rans_decode` only when returned bad is 0. In that success branch, decoder-state input fields are exactly $(\mathsf{count}, \mathsf{size}, m) = (1024, \mathsf{encoded_size}, 13)$. The current harness-control theorem concludes only

$$bad_{\mathrm{enc}} \in \{0, 1\}, \qquad \mathsf{decoder_ran} \iff bad_{\mathrm{enc}} = 0$$

and this field initialization. That control theorem is not itself an inverse. Week 13's separate semantic theorem sequentially invokes verified theorem 3.1, verified theorem 3.2, and the actual-copy theorem inside this module to close the following conclusion.

Verified theorem 3.3 (Actual encoder–copy–decoder core inverse)**.** `actual_rans_encode_copy_decode_inverse` assumes only initial argument bindings and `mode2_hbz_symbol_stream`. In a terminating execution, actual encoder failure leaves the decoder unexecuted and preserves its initial array/state. On actual encoder success, the copied buffer becomes an exact `trace_bytes` input and the decoder satisfies

$$bad_{\mathrm{dec}} = 0, \qquad off_{\mathrm{dec}} = \mathsf{encoded_size}, \qquad decoded[0..1024) = symbols[0..1024)$$

while preserving the $[1024, 2048)$ tail. Encoder success, copied-region equality, decoder success, and output equality are not preconditions. This is Hoare partial correctness; success reachability and termination of both loops remain separate obligations.

Definition 3.16 (Production full-HBZ harness)**.** `SignaturePackUnpackHbzFullHarness.run` calls actual production `SignaturePackMode2Target.M._encode_hb_z1_full`, then calls actual `SignatureUnpackMode2Target.M._decode_hb_z1_full` only when the returned size is not 0. `HbzFullActualHarness.run` executes the same control pattern with focused full-HBZ extractions. Week 14's relational theorem exactly aligns the result tuples and `decoder_ran` flags of the two harnesses.

Verified theorem 3.4 (Production full-HBZ success-conditioned inverse)**.** `signature_pack_unpack_hbz_full_actual_exact` proves that the two harnesses return equal values and agree on decoder execution from the same initial arguments. The subsequent `signature_pack_unpack_hbz_full_inverse_mode2` assumes only initial argument bindings and `canonical_hbz_mode2`. For a terminating execution, $size = 0$ leaves the decoder unexecuted and preserves the initial encoded/decoded/bad arrays. When $size \neq 0$,

$$4 \leq \mathrm{uint}(size) \leq 1024, \qquad bad = 0, \qquad decoded[0..1024) = hbz_0[0..1024)$$

and the coefficient tail, encoded-suffix frame, and actual `prepared_symbols` trace witness are preserved. Encoder success, decoder success, output equality, and the trace are not preconditions. This is Hoare partial correctness; reaching nonzero size and termination are separate obligations.

Boundary statement 3.1 (Success-conditioned HBZ)**.** $\mathrm{CanonicalHBZ}_2(h)$ guarantees success of the prepare leaf, but not that the actual full encoder with its fixed 1024-byte rANS workspace returns nonzero size. Week 14's production theorem does not assume encoder or decoder success and retains the actual return's $\mathsf{size} = 0$ branch in its conclusion. Week 15 excludes that failure branch only for terminating fixed all-zero-input executions, not for general canonical inputs or termination itself.

Verified theorem 3.5 (Production success for fixed all-zero HBZ)**.** The only preconditions of `signature_pack_unpack_hbz_zero_success_mode2` bind the initial encoded/decoded/bad arrays and the exact `zero_hbz` input. A terminating execution satisfies

$$size \neq 0, \qquad 4 \leq \mathrm{uint}(size) \leq 1024, \qquad \mathsf{decoder_ran}, \qquad bad = 0$$

with all first 1024 recovered coefficients equal to 0, preserving the coefficient/encoded tail frames and exact `mode2_trace_bytes(all_six_symbols)`. The subtheorem `actual_rans_encode_all_six_success` shows that nonzero *bad* from the actual encoder requires a normalization trace exceeding 1020 bytes, contradicting the all-6 trace's 1020-byte bound. This is fixed-input Hoare partial correctness, not termination/losslessness or a phoare = 1 theorem.

Definition 3.17 (Actual KeyGen matrix/finalizer snapshot)**.**

`ActualM23MatrixFinalizeSnapshot.run` stores the returned *bp* from actual `_kp_m23_matrix` as pre_bp, then calls actual `_keypair_finalize_m23`. At every active coefficient,

$$\begin{aligned} r_i &= (\mathsf{pre_bp}_i + s_{2,i} + a_i) \bmod q, \\ r_i &= 2b_{1,i} + b_{0,i}, \qquad -1 \le b_{0,i} \le 1, \\ s'_{2,i} &= s_{2,i} - b_{0,i} \end{aligned}$$

defines `actual_snapshot_low_high_decomposition`. Over the same active range, `actual_snapshot_mod2q_zero` requires

$$2(a_i - 2b_{1,i}) + 2\mathsf{pre_bp}_i + 2s'_{2,i} \equiv 0 \pmod{2q}$$

the displayed equality.

Verified theorem 3.6 (Semantic composition of actual KeyGen snapshots)**.** `actual_m23_matrix_finalize_semantic_snapshot` assumes only exact bindings of six initial arrays, matrix/input representation ranges, active $s_2 \in [-1, 1]$, and active $a \in [0, q)$. Its conclusion combines the matrix output/NTT representation, exact finalizer semantics, the two predicates of definition 3.17, and initial–snapshot–final tail frames. Neither finalizer output nor the KG equation is a precondition.

Boundary statement 3.2 (`STOP-KG-NTT` at paper freeze)**.** The pre_bp in verified theorem 3.6 is actual `output_row`. Target 77's `output_row_from_mode2_ntt_words` expands it into the actual full-invNTT/pointwise-word representation, and `actual_m23_matrix_snapshot_rows_explicit` transports it to the two actual harness rows. Both results compile, but do not identify that expression with the security model's polynomial product $A_{\text{gen}}s_{\text{gen}}$. Odd-root orthogonality/full-NTT convolution and the `Rq.poly`-to-security-list multiplication adapter are absent from the frozen targets, so the snapshot congruence cannot be promoted to the paper's $As = qj \pmod{2q}$. This audit stopped the frozen paper's KeyGen frontier at KG-2/finalization.

Verified theorem 3.7 (Post-freeze Rq–HAETAE representation preservation)**.** `RqHAETAEBridge.rq_poly_repr` requires only HAETAE polynomial well-formedness and `incoeff` equality on the active 256 coefficients. For this relation, `rq_poly_add_repr`, `rq_poly_mul_repr`, `rq_poly_dot3_repr` preserve addition, checked negacyclic multiplication, and 3-term row products, respectively. `Mode2KeygenNttMulBridge`'s `keygen_mode2_row_product` and `mode2_row_product_haetae_dot` connect actual mode-2 NTT output to this relation. The desired $As = qj$ equation appears in none of the premises.

Boundary statement 3.3 (Latest `STOP-KG-AVEC-QJ`)**.** The post-freeze actual-sampler theorem interprets terminating `avec` through SHAKE128 bytes with nonces 515/516, little-endian 16-bit candidates, and < 64513 rejection, connecting it to the paper's *a*. The paper's *qj* is a separate basis correction with $q = 64513$ only at row 0, coefficient 0. In contrast, the current security model's named *qj* is a synthetic seed generator, always giving 401 at that position. `security_named_qj_is_not_paper_qj` compiles this inequality, so the two objects cannot be identified. The first repair obligation is `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS`.

Definition 3.18 (μ prefix)**.** Let μ_S^{64} be Sign's 64-byte array and μ_V^{32} Verify's 32-byte array. Then

$$\mathrm{MuPrefix}_{32}(\mu_S^{64}, \mu_V^{32}) \iff \forall i \in [0, 32),\ \mu_S^{64}[i] = \mu_V^{32}[i].$$

that is, $\mathrm{take}_{32}(\mu_S^{64}) = \mu_V^{32}$.

Definition 3.19 (Raw branch and valid range)**.** For 64-bit words p, ℓ, the two code predicates are

$$\mathrm{RawPreLen}(\ell) \iff 0 \le \mathrm{uint}(\ell) < 2^{63}, \qquad \mathrm{ValidRegion}(p, \ell) \iff \mathrm{uint}(p) + \mathrm{uint}(\ell) \le 2^{64}$$

The first selects the raw branch in Verify's actual branch on `prelen>>63`; the second excludes wraparound in address addition.

3.2 Hoare statements and relational equivalence

The EasyCrypt statement

$$\mathrm{hoare}[P : \Phi \Longrightarrow \Psi]$$

does not assert termination of P. It states partial correctness: every execution starting in Φ that **terminates** satisfies Ψ. Losslessness or almost-sure termination of KeyGen's retry loop, and equality with the paper's distribution, therefore require separate theorems.

The relational statement

$$\mathrm{equiv}[P \sim Q : \Phi \Longrightarrow \Psi]$$

means that results of P, Q, started in states satisfying the pre-relation Φ, satisfy post-relation Ψ. The hash theorems here do not replace SHAKE with an abstract function; they directly compare the two sides' generated absorb/finalize/Keccak/squeeze procedures in this relational logic.

3.3 Exact traces and accepted tails

The trace modules of the new raw-API theorems copy actual procedure control flow while adding only ghost observations such as `observed_mu`, pointers, lengths, and `tail_reached`. Exact equivalence means that actual procedure and trace yield the same final `Glob.mem` and return value, not that every execution reaches the hash tail.

Verify can reject before passing signature-length, unpacking, norm, and other checks. The current theorem exposes this branch by proving

$$\mathrm{VerifyAccept} \Longrightarrow \mathrm{TailReached}$$

and only then, for successful executions, connects the VK/pre/message addresses and lengths passed to actual `__verify_hash_mu` to the observations. This is distinct from claiming that every signature reaches the tail.

Preservation of the actual procedure's result by an exact trace does not automatically place the internal hash-call return value and post-execution ghost fields in a relational theorem. The remaining **OBL-MU-PRODUCT-REPLAY** is the rule connecting `observed_mu` in two completed sequential traces to the existing pRHL hash-return theorem. Simple `sim`, `seq`, and `byequiv` cannot eliminate the different residual continuations, so this edge is explicitly **DEFERRED**.

3.4 Array capacity and logical length

The extracted theories use fixed arrays capable of supporting several parameter sets: `BArray2080`, `BArray2752`, `BArray2948`. This does not mean that mode-2 logical lengths are 2080, 2752, and 2948 respectively. Actual mode-2 API lengths are 992 bytes for the verification key, 1408 for the secret key, and 1474 for the signature; the current theorems track the relation on the first 992 bytes. Separately, signature-codec results track the layout and round trip of 32 challenge bytes and 1024 low bytes, the first 1056 signature bytes. In HBZ leaves, `BArray8192` holds 2048 32-bit coefficients and symbol `BArray2048` holds 2048 bytes, but the mode-2 logical count is 1024. Week 8 frame theorems therefore explicitly preserve the $[1024, 2048)$ tails of both arrays.

4 The proof chain closed so far

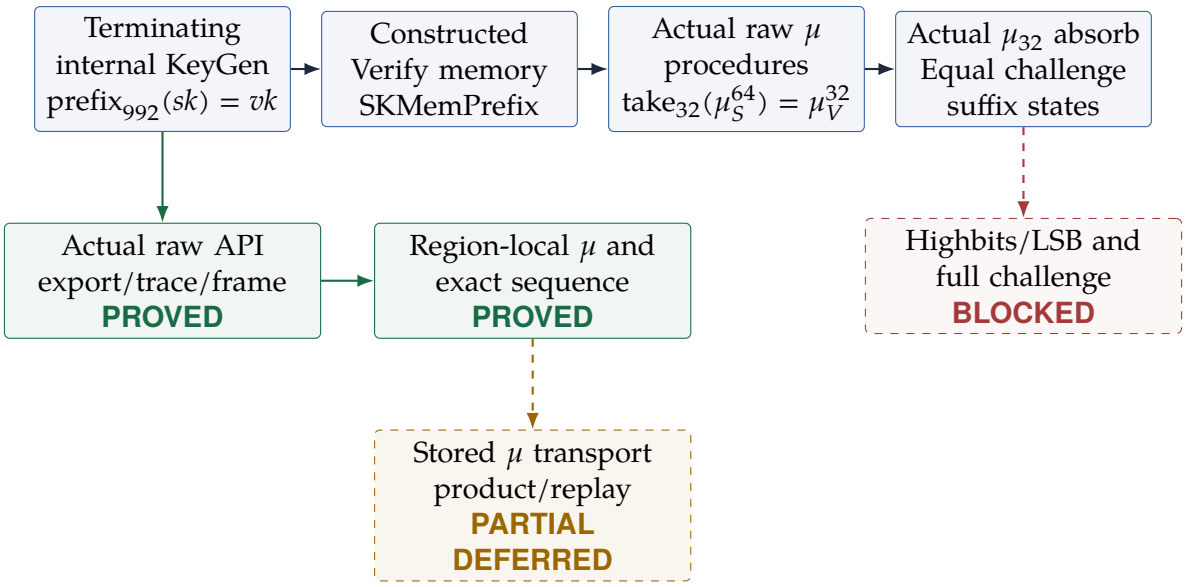


Figure 1: The verified raw/internal chain (above) and the Week 6/7 API semantic boundary (below). This security boundary remains unchanged in Weeks 8–9.

4.1 Step 1: the packed-key prefix returned by KeyGen

Jasmin `_pack_sk_m23` first copies `vkp[0..vkbytes)` to `skp`. In mode 2, `vkbytes=992`, and subsequent writes follow this layout.

$$\underbrace{[0,992)}_{vk} \parallel \underbrace{[992,1184)}_{3\cdot 64\text{-byte } \eta \text{ vector}} \parallel \underbrace{[1184,1376)}_{2\cdot 96\text{-byte } \eta_2 \text{ vector}} \parallel \underbrace{[1376,1408)}_{32\text{-byte key}}.$$

`pack_vec_eta_to_prefix_frame` and `pack_vec2_eta_to_prefix_frame` prove that later writes do not change $[0, 992)$.

Verified theorem 4.1 (Generated packer prefix)**.** `pack_sk_m23_mode2_vk_prefix` proves that, under mode-2 constants, if the actual generated `_pack_sk_m23` terminates, then

$$\text{VKPrefix}_{992}(res, vk_0)$$

holds. Through `keypair_full_m23_mode2_return_prefix` and `keypair_internal_mode2_return_prefix`, this result lifts to the return values of actual generated `_keypair_full_m23` and `crypto_sign_keypair_internal_mode2_jazz`.

This theorem holds without analyzing the distributions inside every KeyGen retry attempt: whichever attempt terminates, the final packer call produces the same prefix. Precisely for this reason, it is a strong return-value invariant but remains partial correctness.

4.2 Step 2: from local arrays to Verify memory

`Mode2KeyMemoryBridge.stored_vk_buffer_at` constructs concrete memory by writing the verification key's first 992 bytes over existing memory using `stores`. `keygen_prefix_memory_relation_is_constructible` uses definition 3.1 to show that this memory satisfies definition 3.3. `keygen_prefix_reaches_mu_memory` converts it to the exact predicate of the hash theorem.

Verified theorem 4.2 (Non-vacuity of raw-hash premises)**.** `generated_raw_mu_preconditions_from_keygen_prefix` chooses *base* = 0, *prelen* = 0, and stored verification-key memory to construct a concrete witness satisfying the no-wrap, raw-branch, valid-region, and `sk_memory_prefix` premises simultaneously.

This result shows that the premises are consistent and reachable from internal KeyGen return values. This witness is historical local evidence constructed with an authored `stores` operation. The Week 5 API-level theorem reaches the same relation using only actual caller memory and generated copy procedures, without using this witness.

4.3 Step 3: from copy helpers to the actual raw KeyGen caller

`easycrypt/refinement/composition/ApiKeyMemoryBridge.ec` directly targets actual generated procedures obtained by focused extraction: KeyGen's `__kp_api_copy_2080_to_addr` and `__kp_api_copy_2752_to_addr`, and Sign's and Verify's `_api_copy_raw_to_2752_prefix`.

Verified theorem 4.3 (Prefix preservation by individual API copy helpers)**.** `keygen_export_vk_mode2_prefix` and `keygen_export_sk_mode2_prefix` prove that terminating actual KeyGen exporters write the 992-byte VK and 1408-byte SK, respectively, to raw memory. `keygen_export_sk_mode2_prefix_frames_vk` additionally proves that the actual SK exporter preserves the existing VK prefix under the minimal `disjoint_regions` premise. `sign_import_mode2_sk_prefix` and `verify_import_mode2_vk_prefix` prove that the actual importers read 1408 bytes and 992 bytes, respectively, from valid input regions, and that the first 992 bytes of both local arrays agree with the original memory. `sign_verify_api_importer_same_inputs` also shows that the two importers return equal values and memory on equal inputs.

The pure adapter theorems `exported_mode2_regions_agree`, `imported_prefixes_agree`, and `imported_sign_sk_reaches_mu_memory` provide logical bridges from prefix agreement of the two exported regions to local Sign/Verify prefix agreement and raw μ memory premises.

In `RawApiAddressBridge.ec`, `canonical_ui64_address_roundtrip`, `mode2_vk_region_bridge`, and `mode2_sk_region_bridge` prove that canonical integer addresses are unchanged through `W64.of_int` and `W64.to_uint` and satisfy both valid-region predicates. Thus the earlier exporter/importer address-representation difference is closed under explicit canonical-region contracts.

In `RawApiKeygenExportComposition.ec`, `keypair_raw_api_exact_export_trace` first establishes equality of final memory and return values between the actual raw KeyGen caller and a trace making the two copies explicit.

Verified theorem 4.4 (Sequential external export by actual raw KeyGen)**.** `keypair_raw_api_exports_matching_prefixes` opens actual generated `cryptolab_haetae_mode2_keypair_internal`. For every terminating execution satisfying the premises of a 32-byte seed read, canonical regions for the 992-byte VK and 1408-byte SK, and separation of the two outputs, it proves that the return value is 0 and

$$\mathrm{ExternalKeyPrefix}(mem_{\mathrm{out}}, sku, vku)$$

holds. In other words, the VK exported first by the actual caller is preserved by the subsequent SK export, and the first 992 bytes of the two external buffers are pointwise equal.

This advances composition of helper facts into one caller postcondition, but remains partial correctness. It does not prove KeyGen retry termination, and canonical/valid regions are calling contracts, not substitutes for physical memory safety.

4.4 Step 4: actual raw Sign/Verify μ procedures

`easycrypt/refinement/composition/ExtractedMuHashPrefix.ec` compares the following generated helpers step by step.

- Sign packed-SK absorb and Verify raw VK-memory absorb;
- The two raw absorbs for equal pre/message addresses and lengths;
- Keccak state initialization, permutation, and finalize;
- The first 32 bytes of Sign's 64-byte squeeze and Verify's 32-byte squeeze.

The resulting `sign_verify_raw_mu_top_wrappers_prefix` proves definition 3.18 at the local wrapper level. The important Week 3 advance was to remove wrappers from the conclusion surface.

Verified theorem 4.5 (Relation between actual generated raw μ procedures)**.** The two procedures of `sign_verify_generated_raw_mu_prefix` are actual generated `Sign._sf_mu_rawpre` and `Verify.__verify_hash_mu`. Under equal pre/message addresses and lengths, `RawPreLen`, two `ValidRegion` predicates, 992-byte `SKMemPrefix`, and address non-wrapping, it proves

$$\mathrm{MuPrefix}_{32}(res_S, res_V)$$

as the result relation.

The Sign side is connected exactly to its wrapper by `sf_mu_rawpre_refines_raw_top`, and the Verify side by `verify_hash_mu_raw_refines_raw_top`; relational transitivity then composes the intermediate theorem. Verify's raw branch is not assumed through a separate flag. `raw_prelen_shr63_zero` proves that the actual `prelen>>63` computation yields 0, selecting that branch.

4.5 Step 5: equality of the challenge suffix

The Sign challenge transcript absorbs $\mathrm{take}_{32}(\mu_S^{64})$ after the 64-byte highbits/LSB part, and Verify absorbs μ_V^{32} at the same position. The current theorem does not yet prove equality of those first 64 bytes; it assumes equal challenge states and starting position 64.

Verified theorem 4.6 (Generated raw μ-to-challenge composition)**.** `generated_raw_mu_to_challenge_suffix_zero_loss` compares two composed programs that execute actual `_sf_mu_rawpre` and actual `__verify_hash_mu`, respectively, followed by the actual Sign/Verify μ_{32} absorb helpers. Under the premises of verified theorem 4.5 and initial challenge position 64, the returned challenge states and positions are exactly equal.

`TranscriptBytes.challenge_input_eq_components` supplies a pure list lemma showing that equality of highbits and LSB bytes, once established, yields equality of the entire input lists. The interface between the current result and the next unresolved obligation is therefore explicit.

4.6 Step 6: exact traces of actual raw Sign/Verify callers

`RawApiCallerMuTrace.ec` exactly compares actual `cryptolab_haetae_mode2_signature_internal` with a Sign trace augmented by ghost observations. `sign_raw_api_exact_mu_trace` preserves final memory and return values; `sign_raw_api_trace_imports_mode2_sk` and `sign_raw_api_trace_binds_hash_inputs` record the imported 992-byte prefix and actual pre/message pointers and lengths.

On the Verify side, `RawApiVerifyMuTrace.ec` connects the full M23, mode 2, internal, raw, and cryptolab wrappers exactly to their traces. Crucially, Verify may reject before hashing.

Verified theorem 4.7 (Verify success reaches the actual hash tail). `verify_raw_api_actual_accept_implies_trace_tail_reached` and `verify_cryptolab_actual_accept_implies_trace_tail_reached` prove that the trace's `tail_reached` is true when the actual return value is 0. `verify_raw_api_actual_accept_binds_hash_inputs` and its cryptolab counterpart bind the VK/pre/message pointers and lengths supplied to actual `__verify_hash_mu` on that successful path to the caller arguments.

This does not prove tail reachability for arbitrary signatures or acceptance of legitimate Sign outputs. Rejecting executions need not reach hashing; tail reachability of legitimate outputs is a separate functional-correctness obligation.

4.7 Step 7: accepted-path generated adapters and the remaining gap

In `RawApiMuReachability.ec`, `raw_api_region_reaches_mu_zero_loss` constructs definition 3.3 from the actual external prefix, Sign import, and canonical VK region. There is no artificial `storesmem0` witness here.

Verified theorem 4.8 (Accepted traces instantiate generated hash/suffix theorems). `raw_api_accepting_execution_hash_mu_zero_loss` proves $\mathrm{MuPrefix}_{32}$ for the results of actual generated `_sf_mu_rawpre` and `__verify_hash_mu`, under observations supplied by the Sign trace and a successful Verify trace. `raw_api_accepting_execution_generated_challenge_suffix_zero_loss` also equates the actual generated suffix-helper results under the same premises and challenge position 64.

However, the program surfaces of these two theorems are the generated hash and suffix helpers, not the trace modules themselves. Week 6 closed the Sign-output frame and region-local memory relations below, but a separate semantic edge transporting the relation to stored results of completed traces is still needed.

4.8 Step 8: Sign-output frames and region-local sequential traces

`RawApiSignOutputFrame.ec` opens the actual output copy and raw Sign caller. `sign_output_copy_exact_and_frames_reused` writes the 1474-byte signature prefix exactly while framing bytes outside it, and `sign_raw_api_frames_reused_regions` lifts this to actual `cryptolab_haetae_mode2_signature_internal`.

Verified theorem 4.9 (Reused-region frame for actual Sign output). If the valid signature/siglen output regions are explicitly disjoint from the VK, SK, pre, and message regions, then every terminating execution returns 0 from Sign, preserves the 992-byte VK, 1408-byte SK, and pre/message regions, and writes 1474 to siglen. `sign_raw_api_preserves_external_key_prefix` also transports KeyGen's `ExternalKeyPrefix` across the actual Sign call. This disjointness contract has a concrete witness, but signing-loop termination is not a conclusion.

`RegionLocalMuEquivalence.ec` and `RegionLocalMuTop.ec` remove the whole-memory equality requirement.

Verified theorem 4.10 (Region-local equivalence of actual generated hashes). `sign_verify_generated_raw_mu_prefix_regionwise` compares actual `_sf_mu_rawpre` and `__verify_hash_mu`. It proves $\mathrm{MuPrefix}_{32}(res_S, res_V)$ using only the mode-2 SK/VK prefix relation, raw branch and canonical/no-wrap regions, and definition 3.5 for pre/message read regions. Unrelated memory bytes need not agree.

Finally, `raw_sign_then_verify_actual_exact_trace` in `RawApiDirectObservedMu.ec` proves that a program executing actual raw Sign followed by actual cryptolab Verify and the sequential execution of the two traces preserve both results and final memory. `sign_then_verify_trace_accept_implies_tail_reached` and `sign_then_verify_accept_binds_observed_inputs` supply tail reachability of successful Verify and both hash descriptors as postconditions.

Frames, byte locality, exact sequence, accept control, and input binding now all compile. Nevertheless, the remaining **OBL-DIRECT-OBSERVED-MU** cannot yet directly compare `SignRawApiMuTrace.observed_mu` and `VerifyCryptolabMuTrace.observed_mu`. This requires **OBL-MU-PRODUCT-REPLAY**, transporting the existing pRHL hash-return theorem to ghost results of completed traces with different residual continuations. Week 7 could not construct this without a new axiom or wrapper widening and froze it as **DEFERRED**. Thus $\delta_{\mu,\text{raw_api_accept}} = 0$ is still not recorded.

4.9 Step 9: the actual mode-2 signature prefix codec

The Week 7 functional-correctness path switched to the actual extracted `_pack_sig_prefix` and `_unpack_sig_prefix`. `pack_sig_prefix_mode2_layout` in `Mode2SignaturePrefixPack.ec` proves that the 256 challenge bits occupy the first 32 bytes and the low signed bytes of the 1024 low coefficients occupy the next 1024 bytes. `unpack_sig_prefix_mode2_layout` in `Mode2SignaturePrefixUnpack.ec` reads this layout in reverse and preserves the unused low-array region $[1024, 2048)$. `Mode2SignaturePrefixRoundTrip.ec` defines an actual-procedure harness calling the two procedures in sequence.

Verified theorem 4.11 (Signature prefix codec)**.** The theorem `pack_unpack_sig_prefix_mode2_roundtrip` calls the two actual generated procedures sequentially. Under definition 3.6, it proves that the 256 decoded challenge entries and 1024 mode-2 low entries equal their inputs, and that the remaining 1024 words of decoded low are preserved. All-zero arrays show non-vacuity of the premises.

This is Hoare partial correctness for the 1056-byte prefix. It excludes size metadata after byte 1056, hbz/h rANS payloads, canonical zero padding, the pack output tail, and the challenge-array tail frame. It concludes neither the full 1474-byte codec nor $\delta_{\text{Encoding}} = 0$.

4.10 Step 10: actual HBZ leaves, table certificates, and pure rANS steps

Instead of inlining **OBL-SIG-HBZ-ENCODE-DECODE** all at once, Week 8 split it into four layers: coefficient/symbol leaves, tables, rANS core, and full wrappers. `Mode2HbzCodecSpec.ec` fixes the mode-2 constants

$$\mathsf{count} = 1024, \qquad m = 13, \qquad \mathsf{offset} = 6, \qquad \mathsf{scale} = 2^{10}, \qquad L = 2^{23}$$

and the predicates of definition 3.7.

Verified theorem 4.12 (Actual HBZ prepare/apply leaf inverse)**.** `encode_hb_z1_prepare_mode2_correct` proves that actual `encode_hb_z1_prepare_jazz` maps canonical coefficients $h_i \in [-6, 7)$ to symbols $h_i + 6$, sets the bad word to 0, and preserves the symbol tail. `decode_hb_z1_apply_mode2_correct` proves that the actual apply procedure restores the original 32-bit coefficients from the same symbols and preserves the coefficient tail. `encode_prepare_decode_apply_mode2_inverse`, which executes the two actual wrappers sequentially, concludes exact recovery of the first 1024 coefficients. The all-zero HBZ array shows non-vacuity of the canonical premises.

This theorem is a local prepare/apply inverse; the rANS buffer, encoder size, and full wrapper do not appear on its program surface. Separately, `Mode2HbzActualBoundary.ec` establishes by exact equivalence that the actual `_encode_hb_z1_full`/`_decode_hb_z1_full` bodies in the focused Week 8 extraction and Week 7 signature pack/unpack extractions are the same procedures. Thus there is no issue of the leaf theorems and full packer referring to different code copies, but procedure identity alone does not supply an inverse.

Verified theorem 4.13 (Actual table certificate and pure rANS inverse)**.**

`Mode2HbzTableCertificate.ec` shows that the 13 frequencies are positive, sum to 1024, and have intervals partitioning $[0, 1024)$. It proves that the actual fast encoder expression computed using reciprocal multiply/high-half/shift and bias equals mathematical E_s within the allowed state range, without 32/64-bit overflow. `actual_mode2_hbz_tables_certified` in `Mode2HbzSymbolWordsGenerated.ec` applies definition 3.8 to the literal encoder/decoder/symbol arrays of actual pack/unpack. Finally, `pure_rans_step_inverse` and `hbz_fast_step_decode_inverse` in `Mode2RansCore.ec` prove

$$D_s(E_s(x)) = x, \qquad 0 \le s < 13, \quad 1 \le x < \mathsf{hbz_xmax}(s)$$

with $s = 0, x = 1$ as a concrete witness to the fast-step premises.

4.11 Step 11: rANS byte stacks, actual suffix copying, and harness control

Instead of immediately connecting the encoder and decoder in lockstep, Week 9 first fixed the shared *xs*/*cuts*/*bytes* semantics of definition 3.9.

Verified theorem 4.14 (Pure normalization traces and word arithmetic)**.** `renorm_len_le2` and `renorm_reduced_bounds` in `Mode2RansByteStack.ec` show that, for $2^{23} \le x < 2^{31}$, each symbol emits only 0, 1, or 2 bytes and the reduced state satisfies $1 \le r < \mathsf{hbz_xmax}(s)$. `renorm_bytes_readback` proves that reading back the segment in decoder order recovers the pre-normalization state; `serialize32_parse_inverse` proves the round trip for the first 4-byte little-endian state. `normalized_fast_step_state_bounds` and `encode_trace_state_bounds` show that every boundary state of a canonical trace remains in $[2^{23}, 2^{31})$, and `valid_rans_trace_canonical` proves that trace objects are constructed from their definitions. Separately, `Mode2RansNormalization.ec` connects actual W32 shifts, low-byte truncation, decoder shift/bitwise-or, and W64 cursor decrements to the same integer expressions. In particular, `encoder_two_byte_decoder_order` verifies that backward stores produce the forward segment $[\mathsf{high}, \mathsf{low}]$.

These theorems are pure/word leaves that do not call the actual generated encoder/decoder. They therefore do not justify inserting `valid_rans_trace` as a precondition on actual output or assuming decoder success.

Verified theorem 4.15 (Actual suffix copying and branching on encoder results)**.** `copy_encoded_suffix_correct` proves for actual `__copy_encoded_suffix` that, if

$$0 \le \mathit{off}_0, \quad 0 \le n, \quad \mathit{off}_0 + n \le 2048$$

then $\mathit{out}[j] = \mathit{enc}[\mathit{off}_0 + j]$ for $0 \le j < n$, and the existing *out* tail in $[n, 2048)$ is preserved. This Hoare partial-correctness theorem opens the actual copy loop, rather than using a constructed decoder buffer.

Of `actual_rans_encode_mode2_control` and `actual_rans_decode_mode2_control`, the encoder theorem fixes the concrete mode-2 tables/count and canonical symbol stream while leaving the initial state arbitrary; the decoder theorem fixes the concrete tables and `mode2_decode_state` input. Each proves only the returned $\mathit{bad} \in \{0, 1\}$. Subsequently,

`actual_rans_harness_branches_on_encoder_result` for definition 3.15 proves the actual encoder–copy–decoder order, decoder non-execution on failure, and exact decoder input fields in the success branch. This conclusion contains no trace equality, decoder success, or symbol round trip.

The exact status at the end of Week 9 was as follows.

Obligation	Status
HBZ prepare/apply, leaf inverse, and bounded tail frames	**PROVED**
Focused/full actual procedure boundary	**PROVED**
Actual 13-symbol table certificate	**PROVED**
Pure and reciprocal fast single-step inverse	**PROVED**
Pure normalization-byte inverse and state bounds	**PROVED**
Actual suffix-copy pointwise/frame theorem	**PROVED**
Actual encoder–copy–decoder harness control	**PROVED**
Actual encoder outer-loop trace refinement	**PARTIAL**
Actual decoder trace-consumption refinement	**PARTIAL**
Actual rANS core inverse and success witness	**PARTIAL**
Success-conditioned full-HBZ inverse	**PARTIAL**

The actual encoder processes symbols in reverse order, stacking normalization bytes at a backward cursor, while the decoder consumes them from the front. Week 9 lacked the two outer invariants establishing this backward-stack/forward- consumption relation between actual $(i, x, off, buffer)$ and pure (x_j, c_j, B). Thus, following boundary statement 3.1, encoder success was not assumed from canonical HBZ alone, and **OBL-SIG-HBZ-ENCODE-DECODE** was **PARTIAL**. This table and `CONTINUE-RANS` are historical verdicts at the end of Week 9.

4.12 Step 12: actual encoder inner semantics and successful sizes

Week 10 further decomposed the actual encoder boundary instead of moving to the decoder. `Mode2RansArrayListBridge.ec` defines the array/list suffixes and segment/frame operations of definition 3.10, and proves the 1024th-suffix base equation, one-element recurrence, canonicality, concatenation, and one-symbol `encode_trace` recurrence.

Verified theorem 4.16 (Array/list suffixes and the actual word step)**.** `encode_trace_suffix_extension` constructs a one-symbol trace from the array's ith symbol and $i+1$th suffix. `actual_mode2_encoder_word_step_correct` in `Mode2RansEncoderWordStep.ec` proves that, under

$$0 \le s < 13, \quad 1 \le \mathrm{uint}(x) < \mathsf{hbz_xmax}(s)$$

the W32 result computed using reciprocal multiplication, shifts, and bias from the actual mode-2 tables equals the W32 representation of pure `hbz_fast_encode_step`. This is the word-level theorem of **OBL-RANS-ENCODER-WORD-STEP**; it is not itself a Hoare theorem for the generated outer loop.

`Mode2RansEncoderInnerProgress.ec` proves that pure normalization length is 0, 1, or 2, along with division/byte addition at each step. Reusable predicates for the actual generated loop are in `Mode2RansEncoderTrace.ec`, and the direct Hoare proof is in `Mode2RansEncoderActualInner.ec`.

Verified theorem 4.17 (Byte/frame steps of the actual generated inner loop)**.** `actual_rans_encode_inner_no_underflow` opens actual `RansEncodeTarget.M._rans_encode` and maintains the local-step invariant of definition 3.11. The actual transition is

$$off \leftarrow off - 1; \qquad encp[off] \leftarrow \text{low8}(x); \qquad x \leftarrow x >> 8$$

and the proof verifies preservation of the written byte segment and lower prefix frame, and absence of underflow in the W64 decrement. The procedure's return postcondition is

$$bad' \neq 0 \quad \vee \quad (bad' = 0 \wedge \text{uint}(off') \leq 1020)$$

The current phase range $0 \leq k \leq 4$ is conservative. Identifying the actual phases with the pure 0/1/2 normalization bytes and connecting the pre-inner-call array to the accumulated outer trace remain **PARTIAL** under **OBL-RANS-ACTUAL-INNER-NORMALIZE**.

Verified theorem 4.18 (Final serialization leaf and actual successful sizes)**.**

`Mode2RansEncoderSerialization.ec` proves that four actual pattern `set8` writes equal `serialize32_le` and preserve exterior and prefix frames. This is a serialization leaf theorem, not yet composed into the generated procedure's full postcondition.

Separately, `actual_rans_encode_success_size_bound` retains the failure branch of actual `_rans_encode` and proves, on success,

$$0 \leq \text{uint}(off') \leq 1020, \qquad 4 \leq 1024 - \text{uint}(off') \leq 1024$$

This is actual-procedure partial correctness for **OBL-RANS-ENCODER-SUCCESS-SIZE**. It does not prove success reachability or termination.

The exact status at the end of Week 10 was as follows.

Obligation	Status
OBL-RANS-ARRAY-LIST-BRIDGE	**PROVED**
OBL-RANS-ENCODER-WORD-STEP	**PROVED** (word level)
OBL-RANS-ACTUAL-INNER-NORMALIZE	**PARTIAL**
OBL-RANS-FINAL-SERIALIZATION	**PROVED** (leaf theorem)
OBL-RANS-ENCODER-SUCCESS-SIZE	**PROVED** (actual partial correctness)
OBL-RANS-ENCODE-REFINEMENT	**PARTIAL**
OBL-RANS-ACTUAL-SUCCESS-WITNESS	**PARTIAL**
OBL-RANS-DECODE-REFINEMENT and core inverse	**PARTIAL**
OBL-SIG-HBZ-ENCODE-DECODE	**PARTIAL**

The remaining encoder postcondition must state simultaneously that the actual returned suffix equals `trace_bytes` applied to `symbol_list_of_array(symbols0)` pointwise, that its length plus the returned offset is 1024, and that the unwritten prefix is framed. This requires maintaining $0 \leq i \leq 1024$, the suffix-trace state, the offset–length equality, `segment_matches`, and `prefix_frame` together in the outer invariant. Week 10 lacked the edge connecting one outer iteration's actual table reads/protection/reciprocal-shift control to the already proved inner-byte and word steps, and composing the final four stores with the accumulated suffix.

Thus the historical Week 10 verdict was `CONTINUE-ENCODER`, and Week 11 targeted only this single edge.

4.13 Step 13: Week 11 actual encoder trace closure

Week 11 directly appended the pure symbol-tail trace to Week 10's local inner invariant and installed an exact invariant. For convenience, write

$$\ell := \text{mode2_normalization_len}(x_0, s),$$
$$W_k := \text{inner_written_suffix}(x_0, s, k), \qquad B_t := \text{encode_trace}(tail).\text{bytes}$$

The central form is then

$$\begin{aligned} I_{\text{tail}}(k) \equiv\ & 0 \le k \le \ell \\ & \wedge\ \text{segment_matches}(encp, off_0 - k, W_k +\!+ B_t) \\ & \wedge\ \text{prefix_frame}(enc_0, encp, off_0 - k). \end{aligned}$$

When the actual guard becomes false, $k = \text{mode2_normalization_len}(x_0, s) \le 2$ is derived. The generated table's $x_{\max}$, reciprocal, bias, packed complement/shift values, and shift ladder are equated with the pure fast step through a dedicated word bridge.

The Week 11 files have the following roles. Read each filename below with the common prefix `Mode2RansEncoder`.

- `TailInvariant.ec`: invariant initialization, one-byte progress, exact loop-exit characterization, and the outer one-symbol success transition;
- `GeneratedWordStep.ec`: connects generated table reads and word updates;
- `SerializationComposition.ec`, `Finalization.ec`, `GeneratedFinalization.ec`: actual final-store composition prepending the serialized state to the accumulated trace;
- `ActualTraceClosure.ec`: Hoare theorem directly opening actual inner/outer loops and final stores;
- `OuterRefinement.ec`: final corollary explicitly expanding the complete successful suffix.

Verified theorem 4.19 (Exact successful suffix of the actual encoder)**.** `actual_rans_encode_trace_closure` directly opens actual generated `RansEncodeTarget.M._rans_encode` to prove the following failure/success postcondition. Below, let

$$B_0 := \text{trace_bytes}(\text{symbol_list_of_array}(symbols_0))$$

Then the postcondition is

$$\begin{aligned} bad' \neq 0 \quad \vee \quad & (bad' = 0 \wedge 0 \le off' \le 1020 \wedge 4 \le 1024 - off' \le 1024 \\ & \wedge\ \text{segment_matches}(encp', off', B_0) \\ & \wedge\ off' + |B_0| = 1024 \\ & \wedge\ \text{prefix_frame}(enc_0, encp', off')). \end{aligned}$$

Success is not a precondition, and the trace is computed canonically from the actual input symbol array. `actual_rans_encode_trace_refinement` is the final corollary expanding the same conclusion. Both theorems compiled in the full 57-target `-no-eco` verification.

The status at the end of Week 11 was as follows.

Obligation	Status
OBL-RANS-ACTUAL-INNER-NORMALIZE	**PROVED**
OBL-RANS-FINAL-SERIALIZATION	**PROVED** (actual composition)
OBL-RANS-ENCODE-REFINEMENT	**PROVED** (success-conditioned partial correctness)
OBL-RANS-ACTUAL-SUCCESS-WITNESS	**PARTIAL**
OBL-RANS-DECODE-REFINEMENT	**PARTIAL**
OBL-RANS-CORE-INVERSE	**PARTIAL**
OBL-SIG-HBZ-ENCODE-DECODE	**PARTIAL**

The end-of-Week 11 verdict was `GO-ENCODER`. This meant that the encoder refinement edge was closed, allowing Week 12 to move to actual decoder trace refinement. It did not mean that encoder termination, success-branch reachability, decoder success, the HBZ round trip, or encoding zero loss had been proved.

4.14 Step 14: Week 12 actual decoder semantic refinement

Week 12 did not treat existence of a pure trace as decoder success; it directly opened both loops of actual generated `RansDecodeTarget.M._rans_decode`. Read the filenames with the common prefix `Mode2RansDecoder`.

- `Cursor.ec`, `CursorSteps.ec`: reference states, segments, initial/one-step/final cursor boundaries, and read ranges;
- `WordStep.ec`, `ActualWord.ec`, `GeneratedStep.ec`: connect concrete `symbol_words` lookups, `dsyms_words` fields, and actual W32/W64 updates to the mathematical decoder step;
- `Normalization.ec`: partial byte replay and exactness of the 0/1/2-byte guards;
- `ActualTrace.ec`: actual outer/inner loop invariants, initial parsing, one-symbol transitions, and exact exits;
- `TopHoare.ec`: top-level Hoare theorem directly targeting the actual generated procedure.

The reference boundaries for the expected symbol list s are

$$c_0 = 4, \qquad c_{i+1} = c_i + |\beta_i|, \qquad c_{1024} = |\mathsf{trace_bytes}(s)|,$$
$$x_0 = \mathrm{ParseLE32}(\mathsf{trace_bytes}(s)[0..4)), \qquad x_{1024} = 2^{23}.$$

`generated_decoder_parse32_from_trace` connects the actual 4-byte parsing in the first equation, and generated-table theorems match the symbol selected by x_i mod 1024 and the reduced state to the mathematical step. The inner invariant maintains

$$k = \mathrm{uint}(off) - c_i, \qquad x = \mathsf{decoder_replay_prefix}(x_{i+1}, s_i, k)$$

The generated guard equals $k < |\beta_i|$, and $|\beta_i| \le 2$, so the actual loop consumes exactly 0, 1, or 2 bytes.

Verified theorem 4.20 (Exact-trace partial correctness of the actual decoder)**.** The preconditions of `actual_rans_decode_trace_refinement` are a canonical 1024-symbol array, exact `trace_bytes` computed from that array, size $4 \le n \le 1024$, state fields $(1024, n, 13)$, the two

actual mode-2 decoder tables, and a pointwise trace-read relation. Neither success nor output equality is a precondition. The postcondition of every terminating execution is

$$bad' = 0, \qquad off' = n,$$
$$decoded'[0..1024) = expected[0..1024), \quad decoded'[1024..2048) = decoded_0[1024..2048)$$

The internal exit fact $x = 2^{23}$ discharges the actual final rejection check, but is not exposed in the return ABI. This theorem directly targets the actual generated procedure; it is not a wrapper or pure-decoder theorem.

The status at the end of Week 12 was as follows.

Obligation	Status
Decoder cursor/table/word-step bridges	**PROVED**
OBL-RANS-DEC-NORMALIZE	**PROVED** (actual-loop connection)
OBL-RANS-DECODE-REFINEMENT	**PROVED** (exact-trace partial correctness)
OBL-RANS-CORE-INVERSE	**PARTIAL**
OBL-RANS-ACTUAL-SUCCESS-WITNESS	**PARTIAL**
OBL-SIG-HBZ-ENCODE-DECODE	**PARTIAL**

Full 65-target `-no-eco` verification passed, and the Week 12 verdict was `GO-DECODER`. The single Week 13 edge was composition transporting the encoder's `segment_matches` and copy-helper `slice_eq` to the decoder's pointwise exact-trace input inside actual `Mode2RansActualHarness.run`. This composition, actual encoder-success reachability, termination of the two loops, full-HBZ wrappers, and encoding zero loss do not follow from the Week 12 theorem.

4.15 Step 15: Week 13 actual rANS core inverse composition

Week 13 closed Week 12's last unconnected edge with two authored theories. `Mode2RansCoreCompositionBridge.ec` supplies the following postcondition adapters.

- `copied_suffix_is_exact_trace`: derives the exact trace at offset 0 of the copied buffer from actual encoder `segment_matches` and actual copy `slice_eq`.
- `segment_matches_implies_exact_decoder_segment_input`: expands the global trace segment into pointwise decoder reads in each normalization segment.
- `actual_decoder_input_from_configured_trace`: constructs Week 12's full decoder-input predicate from the three actual state stores and the copied trace.
- `encoder_success_size_word_bridge`: proves non-wrapping `W64` subtraction and equality with the integer trace size from the actual successful-offset range.

Verified theorem 4.21 (Success-conditioned core inverse of three actual procedures)**.** `actual_rans_encode_copy_decode_inverse` in `Mode2RansCoreActualInverse.ec` directly targets `Mode2RansActualHarness.run`. Its only preconditions are bindings of six initial arguments and canonical `mode2_hbz_symbol_stream`; it does not require encoder success. Every terminating execution satisfies one of the following two branches.

$$bad_{\text{enc}} \neq 0 \implies \neg\text{decoder_ran} \wedge decoded' = decoded_0 \wedge state' = state_0,$$
$$bad_{\text{enc}} = 0 \implies \text{decoder_ran} \wedge bad_{\text{dec}} = 0 \wedge off_{\text{dec}} = \text{encoded_size}$$
$$\wedge\ decoded'[0..1024) = symbols[0..1024).$$

The success branch also preserves decoder input fields $(1024, \mathsf{encoded_size}, 13)$, the output $[1024, 2048)$ tail frame, the actual encoder's exact suffix, and the initial prefix frame. Thus there is no circular assumption repeating the copied trace or decoder results as premises.

The theorem's premises are satisfiable by a canonical all-zero symbol array, but its conclusion branches on actual encoder failure/success, so success-branch reachability does not follow. The composition module is also a proof-authored harness directly calling three fixed generated procedures, not the production `_encode_hb_z1_full`/`_decode_hb_z1_full` wrappers themselves.

The completion status for Week 13 was as follows.

Obligation	Status
Copy-trace/pointwise decoder-input/W64 size adapters	**PROVED**
OBL-RANS-CORE-INVERSE	**PROVED** (success-conditioned partial correctness)
OBL-RANS-ACTUAL-SUCCESS-WITNESS	**PARTIAL**
Encoder/decoder termination	**PARTIAL**
OBL-SIG-HBZ-ENCODE-DECODE	**PARTIAL** (actual full-wrapper composition)

Full 67-target `-no-eco` verification and independent theorem-surface checks passed, and the Week 13 verdict was `GO-CORE`. The single Week 14 implementation edge was success-conditioned composition of the already closed prepare/apply inverse and this core inverse at the actual full-HBZ wrapper boundary. Actual success witnesses, termination, the h codec, the full 1474-byte codec, and encoding zero loss do not follow from the Week 13 theorem.

4.16 Step 16: Week 14 production full-HBZ wrapper composition

Week 14 closed the production boundary left by Week 13 with five authored theories. `Mode2HbzInternalBoundaries.ec` connects the prepare, rANS encode/decode, and apply procedures in full-wrapper and existing focused extractions by exact equivalence. The following two adapters connect the coefficient and symbol boundaries.

- `prepared_hbz_implies_mode2_symbol_stream`: transports actual prepare results on canonical HBZ to the Week 13 core inverse's 13-symbol-stream premise.
- `decoded_prefix_preserves_prepared_hbz`: returns the symbol prefix recovered by the actual decoder to the apply theorem's prepared-HBZ premise.

`Mode2HbzFullEncodeTrace.ec` and `Mode2HbzFullDecodeInverse.ec` directly target the actual full encoder and decoder, respectively. The former concludes actual prepare-local symbols and the exact encoded trace, size, and suffix frame; the latter concludes $bad = 0$, the original 1024 coefficients, and the coefficient tail frame from that trace. Both results are composed in the direct harness of `Mode2HbzFullActualInverse.ec`.

Verified theorem 4.22 (Production full-HBZ wrapper inverse)**.** The boundary lift in `Mode2HbzSignatureBoundaryLift.ec` proves exact equality of result tuples and the `decoder_ran` flag between the following two executions.

- production: `SignaturePackUnpackHbzFullHarness.run`

- focused: `HbzFullActualHarness.run`

The exact-equivalence theorem is `signature_pack_unpack_hbz_full_actual_exact`. Its Hoare corollary `signature_pack_unpack_hbz_full_inverse_mode2` assumes only canonical HBZ and initial argument bindings and shows the following.

$$\begin{aligned} size = 0 &: \neg\mathsf{decoder_ran} \text{ and initial-buffer preservation,} \\ size \neq 0 &: 4 \le \mathrm{uint}(size) \le 1024,\ \mathsf{decoder_ran},\ bad = 0, \\ &\quad decoded[0..1024) = hbz_0[0..1024) \text{ and two tail frames.} \end{aligned}$$

The success branch also supplies actual prepare's `prepared_symbols` and exact `mode2_trace_bytes` witnesses. This is not circular composition: neither success nor decoder output is required as a precondition.

The completion status for Week 14 was as follows.

Obligation	Status
Full/focused internal-procedure equivalence and coefficient–symbol adapters	**PROVED**
Direct refinement of actual full-HBZ encoder/decoder	**PROVED**
OBL-SIG-HBZ-ENCODE-DECODE	**PROVED** (success-conditioned partial correctness)
OBL-RANS-ACTUAL-SUCCESS-WITNESS	**PARTIAL**
Encoder/decoder termination	**PARTIAL**
OBL-SIG-H-ENCODE-DECODE	**SPECIFIED**

Full 72-target `-no-eco` verification, including the five new full-HBZ theories, passed, and the Week 14 verdict was `GO-HBZ`. The next single implementation obligation was to show within EasyCrypt that all-zero HBZ coefficients—an all-6 symbol stream after prepare—reach the actual nonzero-size success branch. Before this witness, the result is not extended to unconditional round trips, the (h) codec, the full 1474-byte codec, or encoding zero loss.

4.17 Step 17: Week 15 actual success postcondition for fixed all-6 input

Week 15 used two authored theories and a minimal strengthening of the existing encoder-closure theorem. `Mode2RansAllSixBudget.ec` supplies the following connections.

- Constructs canonical HBZ and the actual prepare's all-6 symbol prefix from actual 32-bit reads of bytewise `zero_hbz`.
- Calculates that symbol 6 emits at most one normalization byte for state $x < 2^{31}$, and that the first four steps emit 0 bytes.
- Hence the 1024-symbol normalization trace has at most 1020 bytes, and the full trace including the final 4 state bytes fits in the fixed 1024-byte workspace.

The existing `Mode2RansEncoderActualTraceClosure.ec` was strengthened. `actual_rans_encode_failure_trace_cause` transports from the existing loop invariant the fact that actual failure requires a normalization trace longer than 1020 bytes. Contradiction with the pure upper bound closes the following actual theorem.

Verified theorem 4.23 (Fixed all-6 production HBZ success). `actual_rans_encode_all_six_success` in `Mode2RansActualSuccessWitness.ec` directly targets actual `RansEncodeTarget.M._rans_encode`, without assuming success or buffer fit as preconditions. On terminating executions, it gives

$$bad = 0, \qquad 0 \le off \le 1020, \qquad 4 \le 1024 - off \le 1024$$

and the exact all-6 trace and prefix frame. `actual_encode_hb_z1_full_zero_success` and `signature_pack_hbz_zero_success_mode2` lift this conclusion to the actual full encoder and production packer. Finally, `signature_pack_unpack_hbz_zero_success_mode2` concludes nonzero size, decoder execution, $bad = 0$, recovery of 1024 zero coefficients, and two tail frames in the production harness on fixed `zero_hbz` input.

The completion status for Week 15 was as follows.

Obligation	Status
Zero-HBZ prepare-to-all-6 and 1020-byte budget	**PROVED**
OBL-RANS-ACTUAL-SUCCESS-WITNESS	**PROVED** (fixed-input Hoare partial correctness)
Fixed zero-HBZ production round trip	**PROVED** (Hoare partial correctness)
Fixed-input and general rANS termination/losslessness	**PARTIAL**

Full 74-target verification, including the two new theories, passed, and the Week 15 verdict was `GO-WITNESS`. This is not `GO-WITNESS-LOSSLESS`. Execution existence, termination, probability-1 success, and success on every canonical HBZ input do not follow. The subsequent one-week minimum paper scope superseded the original (h)-codec recommendation, deferring (h).

4.18 Step 18: Week 16 MINCORE KeyGen snapshot

The first Week 16 task was transparent composition of two actual helpers, rather than the full KeyGen or API. `Mode2KeygenSnapshotAlgebra.ec` proves, as pure integer theorems, for canonical residue r,

$$r = 2\,\mathrm{high}(r) + \mathrm{low}(r), \qquad -1 \le \mathrm{low}(r) \le 1$$

and mod-$2q$ congruence of raw sums/residues. `ActualM23MatrixFinalizeSnapshot.run` in `Mode2KeygenCoreEquation.ec` directly calls actual `_kp_m23_matrix` and `_keypair_finalize_m23` in sequence, preserving the intermediate bp as `pre_bp`.

Verified theorem 4.24 (Actual matrix/finalizer semantic snapshot). `actual_m23_matrix_finalize_semantic_snapshot` assumes only input/matrix bounds and initial array bindings, and jointly concludes the following.

- Actual matrix-helper `mode2_output_repr_bound16` and `mode2_ntt_repr_bound24`;
- Exact array/HAETAE semantics of the actual finalizer and $r = 2b_1 + b_0$, $-1 \le b_0 \le 1$, $s_2' = s_2 - b_0$;
- For every active coefficient, $2(a - 2b_1) + 2\,pre_bp + 2s_2' \equiv 0 \pmod{2q}$;
- Initial-to-snapshot-to-final, NTT scratch, and s_2 tail frames.

Neither final outputs nor the desired KG predicate appear as preconditions.

However, `pre_bp` is only a full-NTT/pointwise-product/inverse-NTT snapshot represented by `KeygenM23ArithmeticSpec.output_row`. There was no theorem identifying it with the paper's polynomial product $A_{\text{gen}}s_{\text{gen}}$, so the status at that time was as follows.

Obligation	Status
Actual matrix/finalizer snapshot semantics	**PROVED**
KG-2-style actual coefficient low/high decomposition	**PROVED**
Adjusted s_2 component of KG-4 / full KG-4	**PROVED** / **PARTIAL**
Faithful KG-1, KG-3, paper $As = qj \bmod 2q$	**BLOCKED**
OBL-MINCORE-SIGN, **OBL-MINCORE-VERIFY**, **OBL-MINCORE-COMPOSITION**	**PARTIAL** / **PARTIAL** / **SPECIFIED**
OBL-SIG-H-ENCODE-DECODE	**DEFERRED**

The pre-change 76-target baseline, including the two KeyGen snapshot theories, passed full verification. The first task's historical verdict was `CONTINUE-KG` and `BLOCKED-KG-NTT-MUL`; the continuation audit below updated it to the final stop verdict.

4.19 Step 19: Week 16 KG-NTT-MUL audit and stop verdict

The follow-up task made the last representation edge of the existing actual NTT chain explicit in the source, without assuming the desired security-model equality. `output_row_from_mode2_ntt_words` in the 77th manifested target, `Mode2KeygenNttMulBridge.ec`, states, under the transformed secret-key representation,

$$\text{output_row} = \text{array256_mont}(\text{full_invntt}(\text{pointwise_row_words}(\cdots)))$$

and `output_row_repr_from_mode2_ntt_words` transports that equality to the existing `wide_slice_repr_bound` contract. `actual_m23_matrix_snapshot_rows_explicit` directly applies this representation to the postconditions of the two active rows, as a consequence of the existing actual two-call snapshot theorem. None of the three theorems mentions or assumes `Rq.&*`, `HAETAE_Algebra.poly_mul`, $A_{\text{gen}}s_{\text{gen}}$, or KG predicates. The file, including these three theorems and the direct harness import, passed individual fresh `-no-eco` compilation.

Boundary statement 4.1 (Two missing KG-NTT-MUL leaves at paper freeze)**.** First, there was no odd-root orthogonality/full-NTT convolution theorem showing, with the actual Montgomery factor,

$$\text{mont}(\text{full_invntt}(\hat{a} \odot \text{full_ntt}(p))) = \text{full_invntt}(\hat{a}) \star_{Rq} p$$

Second, there was no adapter transporting products of native `Rq.poly` arrays to the security model's integer-list `poly_mul/matrix_vec_mul`. Legacy `Rq.ntt` differs from the actual full transform and cannot fill this gap. Thus renaming the snapshot's `pre_bp` as $A_{\text{gen}}s_{\text{gen}}$ is circular.

The frozen verdict is `STOP-KG-NTT`. KG-2/finalization is **PROVED**; only the adjusted s_2 component of KG-4 is **PROVED**; faithful KG-1/KG-3, full KG-4, and the paper equation were not proved. The KeyGen frontier is frozen at this boundary. The 78th target, `Mode2SignAcceptedCore`, subsequently compiled direct calls to the three actual helpers and accepted-branch control. However, the full challenge-semantics leaf is absent, so the verdict is `STOP-SIGN-CHAL-MODE2`, and S-1–S-7 are unproved. The four subsequent Verify targets

compiled sequential control of the five actual helpers, exact machine-word expressions for V-1/V-2/V-5/V-6, the W64 norm gate, and tail/mismatch word expressions. With no matrix CRT semantic bridge, the verdict is `STOP-VERIFY-MATRIX-CRT`; at freeze time, the first leaf for resuming work was `rq_mul_coeff_foldr_to_bigi`, followed by the 2-by-4 NTT/CRT procedure leaves.

4.20 Step 20: paper freeze and post-freeze KeyGen semantic alignment

Commit `4bde1207e2a290952d491d5b41c4bd773081e3c4` added no new theorems; it aligned the 82 authored targets, claim ledger, theorem graph, reproduction procedures, and paper PDF at the same `PAPER-FROZEN` boundary. The frozen PDF's SHA-256 is `be935948028829556951863b44ceb2e6c5b2037820991b3a43a4b461b036e53d`. Subsequent work proceeded in the external NTT tree, the existing 77th KeyGen target, and `post-freeze/` auxiliary targets without retroactively changing this paper claim matrix.

The first auxiliary layer closes the pure NTT and representation edges blocked at paper freeze. `haetae-ntt-verify/easycrypt/NTTFullSpectralAction.ec` proves `rq_mul_coeff_foldr_to_bigi`, `full_ntt_montgomery_spectral_action`, and the general `full_ntt_montgomery_row_product`. It does not use legacy `Rq.ntt`, which differs from the actual full transform. `NTTMode2RowSpecializations.ec` specializes these results to the mathematical 2-by-3 KeyGen and 2-by-4 Verify row products.

`RqHAETAEBridge.ec` introduces the coefficientwise representation relation `rq_poly_repr`. Its only premises are well-formedness of HAETAE polynomials and equality of active coefficients; they do not include the desired convolution or KG equation. The following theorems were freshly compiled.

- `rq_poly_add_repr`: representation preservation for `Rq.&+` and `HAETAE_Algebra.poly_add`;
- `rq_poly_mul_repr`: representation preservation for checked negacyclic `Rq.&*` and `HAETAE_Algebra.poly_mul`;
- `rq_poly_dot3_repr`: transports the sum of three products to `HAETAE_Algebra.poly_dot`;
- `keygen_mode2_row_product` and `mode2_row_product_haetae_dot`: connect actual KeyGen NTT output to the native Rq row product and HAETAE three-term dot product.

Thus the two pure odd-root/full-NTT and Rq–HAETAE leaves missing at paper freeze are no longer globally unresolved obligations. However, this alone does not automatically identify actual matrix words, security-side A, secret/error packaging, and the paper's mod-$2q$ equation.

The second auxiliary layer aligns the deterministic semantics of terminating actual mode-2 `avec` expansion with the paper objects. `post-freeze/KgActualAvecQjSemantics.ec` proves the following.

- The two row nonces are 515 and 516, respectively, and the SHAKE128 input appends two nonce bytes to the 32-byte seed;
- Domain byte 31, final padding byte 128, 168-byte squeeze blocks, and the 672-byte size of the first four blocks match actual framing;
- Adjacent bytes are read as little-endian 16-bit candidates, only values below 64513 are accepted, and accepted coefficients are stored without further modular reduction;
- The two 256-coefficient rows obtained through this process are the paper-level mode-2 vector a.

In the paper's $j = (1,0)^T$, the first 1 is the identity constant polynomial, so qj adds $q = 64513$ only to coefficient 0 of row 0, with all other coefficients zero. `paper_qj_row0_coeff0`, `paper_qj_row0_other`, and `paper_qj_remaining_row` separate these cases.

The two auxiliary theorems in `post-freeze/KgActualAvecQjComposition.ec`,

`mode2_sampler_facts_actual_paper_a`, `actual_snapshot_zero_adds_paper_qj`

compose these two paper objects with actual sampler facts and the existing snapshot mod-$2q$ equation. The top-level Hoare theorem

`checked_mode2_parent_m23_finalize_actual_avec_qj`

exposes both results for terminating executions of the checked sampler, actual matrix helper, and finalizer. Sampler termination and distribution are not conclusions.

Boundary statement 4.2 (`STOP-KG-AVEC-QJ`)**.** The existing security model's `haetae_mode23_qj_vector` is neither actual ExpandVecA nor paper qj, but a synthetic arithmetic generator using tag 401 in `public_key_seed_polyveck`. Compiled theorems calculate coefficient 0 of its rows 0/1 as 401 and 403, respectively, for every seed, whereas paper $qj[0][0] = 64513$. `security_named_qj_is_not_paper_qj` directly proves this inequality. An independent FIPS-202 all-zero-seed trace also gives 44985 for the actual first coefficient of nonce-515 row 0, differing from the synthetic value 401. Assuming the desired identity would concretely require $44985 = 401$ and is therefore unsound.

Thus the latest minimal repair obligation is `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS`. The security-side `public_rounding_vector_seed` must be replaced or refined with actual SHAKE128, little-endian parsing, < 64513 rejection, and nonce 515/516 semantics; the paper qj correction must remain a separate mod-$2q$ object, and an actual-array representation must then be proved. Even afterward, the current HAETAE algebra's mode-2 width-4/mod-q representation must be connected to the paper augmented matrix's width-$1+3+2 = 6$/mod-$2q$ packaging. This step therefore claims neither KG-1–KG-4, $As = qj \pmod{2q}$, sampler distribution, termination, nor $\delta_{\text{KeyGen}} = 0$.

5 Exactly what has been established for security

5.1 The meaning of local zero loss

verified theorem 4.6 uses neither an approximation between probability distributions nor a random-oracle idealization. It is a relational equivalence stating that two deterministic generated procedures return the same state under the same premises. Thus, for this particular seam only, we may record

$$\delta_{\mu,\text{raw_top}} = 0$$

as an equality.

Boundary statement 5.1 (Equalities that cannot be promoted)**.** The local equality implies none of

$$\delta_{\text{Sign}} = 0, \qquad \delta_{\text{Verify}} = 0, \qquad \delta_{\text{Encoding}} = 0$$

these equalities. The verification script fails if strings expressing such unjustified promotions appear in the authored tree.

Although the name of verified theorem 4.8 contains `zero_loss`, it does not currently justify an API equality. The cross-call frame, region-local hash relation, and exact sequential trace now

compile, but there is no product/replay conclusion transporting the generated hash return value to the stored observation in the completed trace. Therefore,

$$\delta_{\mu,\text{raw_api_accept}} = 0$$

is not yet recorded.

This distinction is essential to avoid understating the loss terms in equation (1). The current theorem only makes one deterministic part inside δ_{Sign} and δ_{Verify} zero; it does not close the entire hybrid, which includes the sampler, rejection, packing, parsing, public context, and API memory. However, many subsidiary API memory boundaries—address round trips, sequential KeyGen export, Sign traces and output frames, accepted Verify tails and input binding, and region-local hash read relations—are now closed by actual caller theorems.

The HBZ leaves/tables/single steps, pure byte stack, actual suffix copy and harness control, and actual encoder inner byte/frame steps and successful-size bounds from Weeks 8–10 are also deterministic functional-correctness evidence. Week 11 closed the full actual encoder successful-suffix equality with the same kind of evidence. Week 12 also closed actual decoder exact-trace recovery as deterministic partial correctness under an exact input relation. Week 13 closed the sequential composition of the three actual procedures as success-conditioned partial correctness. Week 14 closed the direct full-HBZ harness including actual prepare/apply, and its exact equivalence at the production SignaturePack/Unpack boundary. Week 15 excluded zero-size failure in terminating production executions on a fixed all-zero input. However, termination and losslessness for that fixed input, success on general canonical inputs, the (h) codec, metadata/padding, and full parsing remain open, so these results cannot be promoted to $\delta_{\text{Encoding}} = 0$.

The Week 16 KeyGen snapshot theorem is also deterministic functional-correctness evidence, but does not mean $\delta_{\text{KeyGen}} = 0$. The final rewrite expanding the actual matrix helper's `output_row` into a full-invNTT/pointwise-word expression and the two active-row consequences for the actual harness compiled as the 77th target. At this 82-target paper boundary, odd-root/full-NTT convolution and the `Rq.poly`-to-security-list adapter were missing, so the boundary was frozen at `STOP-KG-NTT`. Sign actual-call control, V-1/V-2/V-5/V-6 for canonical decoded (x, v, h, c), the W64 norm gate, and the tail/mismatch word expressions also appear in the frozen claim matrix, but the matrix/CRT and challenge blockers remain open.

The post-freeze auxiliary proofs closed pure NTT spectral action, the native Rq row product, and Rq–HAETAE representation preservation, interpreted terminating actual `avec` executions as the paper's a, and composed the snapshot with a separate paper-qj term. However, coefficient 0 of the existing security-side object is 401, the paper's qj is 64513, and the first coefficient of the actual all-zero-seed trace is 44985. This mismatch is fixed by the compiled `security_named_qj_is_not_paper_qj`. Thus the current auxiliary frontier is `STOP-KG-AVEC-QJ`; the sampler, retry, packer, public API, distribution/termination, and final KG equation remain outside it.

5.2 Current claim matrix

Table 2: Main claims and their exact current strength

Claim	Status	Established meaning / remaining boundary
KeyGen packed prefix	**PROVED**	For terminating mode-2 generated KeyGen returns, $sk[0..992) = vk[0..992)$; termination and distribution are separate

Claim	Status	Established meaning / remaining boundary
KeyGen-to-memory reachability	**PROVED**	Constructs a concrete local Verify memory witness from internal return values; a historical seam unused by the Week 5 API theorem
Individual public API key copy helpers	**PROVED**	Partial correctness of 992/1408-byte prefix preservation by the actual generated exporters/importers
Canonical address bridge	**PROVED**	Round trips between integer ABI addresses and `W64` exporter/hash addresses, with 992/1408/1474-byte region specializations
Actual raw KeyGen export	**PROVED**	If the actual caller terminates, the first 992 bytes of the disjoint external VK/SK regions agree; retry termination is separate
Actual raw Sign trace	**PROVED**	Preserves the actual caller's return value/memory while exposing SK import and hash pointer/length observations
Actual Verify accepted path	**PROVED**	Raw/cryptolab exact traces and *accept* ⇒ *tail*; actual hash input binding on success
Raw-helper μ	**PROVED**	For μ, relations for actual absorb, Keccak, finalize, and 64/32-byte squeeze operations
Generated top control	**PROVED**	Directly compares the actual `_sf_mu_rawpre` and the raw branch of `__verify_hash_mu`
Challenge suffix	**PROVED**	Equality of state and position after actual μ_{32} absorption beyond position 64
Accepted generated adapters	**PROVED**	Caller trace facts instantiate the actual generated hash-call μ prefix and position-64 suffix helper theorems
Sign-output frame	**PROVED**	Partial correctness showing that actual raw Sign's 1474-byte signature and 8-byte siglen writes preserve disjoint VK/SK/pre/message regions
Region-local generated μ	**PROVED**	Proves the prefix relation of generated hash μ returns using only actual VK/pre/message read regions, without whole-memory equality
Actual Sign-to-Verify sequence	**PROVED**	Result/final-memory equivalence of the two actual callers and sequential traces, accept-to-tail, and post-execution hash descriptor binding
Accepted raw API μ composition	**PARTIAL**	Only direct trace `observed_mu` equality remains; frame/locality/control are closed
Product/replay transport	**DEFERRED**	No justified rule transports the pRHL result for generated hash returns to stored `observed_mu` in completed traces
Signature prefix codec	**PROVED**	Actual pack/unpack round trip for the 32-byte challenge and 1024-byte low part; partial correctness under canonical inputs
Signature prefix frame	**PARTIAL**	The unpack low-array tail is preserved, but the pack signature tail and challenge-output tail remain open
HBZ prepare/apply leaf	**PROVED**	Actual leaves map canonical 1024 coefficients to a 13-symbol alphabet and restore them, preserving bounded array tails
HBZ actual procedure boundary	**PROVED**	HBZ full procedures in focused/full extractions and actual signature pack/unpack extractions are exactly equivalent; the inverse is separate

Claim	Status	Established meaning / remaining boundary
HBZ table certificate	**PROVED**	Actual encoder/decoder/symbol arrays satisfy positive frequencies, a 1024-slot partition, and reciprocal arithmetic certificates
Pure rANS single step	**PROVED**	Mathematical $D_s(E_s(x)) = x$ and agreement with the actual reciprocal fast step within the allowed range; excludes normalization bytes and loops
Pure rANS byte stack	**PROVED**	0/1/2-byte normalization readback, little-endian state parsing, *xs*/*cuts*/*bytes* construction, and preservation of $[2^{23}, 2^{31})$ state bounds; excludes generated loops
Actual encoded-suffix copy	**PROVED**	Pointwise slice equality and unused-tail frame for actual `__copy_encoded_suffix`; Week 13 derives integer input bounds and W64 non-wrapping from actual encoder success for use in core composition
Actual rANS harness control	**PROVED**	Actual encoder–copy–decoder calls, failure/success branching on the encoder return, and decoder-field initialization; this existing control theorem itself has no inverse conclusion
Array/list suffix bridge	**PROVED**	Exact 1024-element list/suffix recurrence, segment/frame operations, and one-symbol trace extension
Actual encoder word step	**PROVED**	W32/W64 reciprocal arithmetic using the actual mode-2 tables equals the pure fast step; word-level equality
Actual inner normalization	**PROVED**	Partial correctness showing that the actual generated loop preserves the pure tail, exact 0/1/2 bytes, cursor, and initial prefix frame, and exits exactly at the actual guard
Final serialization	**PROVED**	Four actual pattern stores are composed at the actual generated procedure exit to prepend the little-endian state to the accumulated normalization trace and preserve the initial prefix frame
Actual encoder success size	**PROVED**	Preserves the failure branch of actual `_rans_encode`; on success, $0 \le off \le 1020$, $4 \le 1024 - off \le 1024$; excludes termination/success reachability
Actual rANS encoder refinement	**PROVED**	Failure/success branches of the actual return *bad*; success-conditioned partial correctness giving full `trace_bytes`, the exact offset–length equality, and initial prefix frame on success
Actual rANS decoder refinement	**PROVED**	Given exact `trace_bytes` for canonical 1024 symbols, actual tables, and state inputs, terminating `_rans_decode` satisfies $bad = 0$, exact consumed size, full symbol recovery, and the output-tail frame; termination/losslessness are separate
Actual rANS core inverse	**PROVED**	In one harness, proves decoder non-execution and initial-state preservation on actual encoder failure, and actual copy plus decoder $bad = 0$, exact consumed size, 1024-symbol recovery, and tail frame on success; success reachability and termination are separate

Claim	Status	Established meaning / remaining boundary
Actual full-HBZ parent	**PROVED**	At the production SignaturePack/Unpack boundary, success-conditioned partial correctness gives zero-size failure preservation or 1024-coefficient recovery on nonzero-size success, size/two tail frames, and the prepare-symbol trace; failure exclusion for fixed zero input is a separate Week 15 theorem, and termination is separate
Fixed all-6/zero-HBZ success postcondition	**PROVED**	Terminating actual all-6 encoder and production zero-HBZ pack/unpack satisfy $bad = 0$, nonzero size, and exact trace/recovery/frames; termination, losslessness, and general-input success are separate
Frozen KeyGen matrix/finalizer snapshot	**PARTIAL**	Proves matrix/NTT expressions for the two actual helpers, finalizer semantics, KG-2-style low/high decomposition, adjusted s_2, and the snapshot mod-$2q$ zero equality; the 82-target paper claim stops at `STOP-KG-NTT`, before faithful KG-1/KG-3/full KG-4 and the paper equation
Post-freeze NTT/Rq–HAETAE bridge	**PROVED**	`rq_mul_coeff_foldr_to_bigi`, full-NTT spectral action, general and mode-2 2-by-3/2-by-4 row products, and preservation of HAETAE coefficient representations by Rq addition, multiplication, and dot products; auxiliary results outside the frozen paper claim matrix
Post-freeze actual a/paper qj	**PROVED**	Partial correctness transporting SHAKE128/LE16/rejection semantics of terminating actual mode-2 `avec` to paper a, and composing a separate basis correction qj with the snapshot; excludes distribution/termination
Security ExpandVecA alignment	**BLOCKED**	The current synthetic `haetae_mode23_qj_vector` differs from actual a and paper qj; `STOP-KG-AVEC-QJ` and `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS`
MINCORE Sign	**PARTIAL**	Direct calls to the three actual helpers and accepted-branch control compile; full challenge semantics and S-1–S-7 headline claims are unproved, hence `STOP-SIGN-CHAL-MODE2`
MINCORE Verify/restricted composition	**PARTIAL** / **SPECIFIED**	Sequential control of the five actual helpers, V-1/V-2/V-5/V-6 machine-word equations, the W64 norm gate, and tail/mismatch word expressions compile; matrix CRT and challenge semantic bridges and restricted composition remain open
Full signature codec	**BLOCKED**	Requires the (h) rANS inverse, metadata, padding, residual frames, and canonical parsing; (h) is **DEFERRED**within the Week 16 scope
Legitimate Sign-output tail reachability	**PARTIAL**	Only the prefix inverse is closed, with progress through the HBZ layer of the full codec; a separate functional-correctness obligation requiring (h), metadata/padding, norm, hint, and response/arithmetic
Public-context μ	**SPECIFIED**	No agreement theorem for the context-length byte, context, and Verify high-bit branch for μ
Full challenge call	**PARTIAL**	The μ suffix and list congruence are available, but highbits/LSB, padding/domain, and squeeze/sampler remain open

Claim	Status	Established meaning / remaining boundary
Full KeyGen/Sign/Verify FC	**SPECIFIED**	Partial progress on the KeyGen snapshot leaf, but no paper-refinement theorems for the three APIs
Implementation EUF-CMA	**SPECIFIED**	Requires all API losses, query accounting, and the exact paper/ROM instance
Paper security reduction	**PARTIAL**	An existing conditional chain is available, but the exact transcript/distribution/forking instances remain open

5.3 Distinguishing assumptions, contracts, and proof obligations

The only cryptographic assumptions that may remain in the final security theorem are the concrete HAETAE-2 MLWE, BST-MSIS, and ROM games. In contrast, the following items must not be hidden as cryptographic assumptions.

- Validity, sizes, separation, and aliasing conditions of public buffers;
- Canonical lengths and mode-2 constants;
- Retry termination/losslessness and RNG/random-tape interpretation;
- FIPS202 byte transcripts, padding, domain separation, and endianness;
- The hyperball sampler and accepted-signature distribution;
- FFT fixed-point errors and the tied-minimum policy.

These are contracts to be proved in API refinement, probabilistic programming, and numerical analysis, respectively. The current authored theories do not cover these gaps with new `axioms` or proof holes.

6 Remaining proof obligations and recommended order

6.1 Actual API boundaries closed in Weeks 5/6

Week 5 closed actual caller control and inputs, and Week 6 closed cross-call memory conditions, within the following scope.

- `RawApiAddressBridge` proves round trips between canonical integer addresses and `W64` addresses.
- `keypair_raw_api_exports_matching_prefixes` proves equality of the first 992 external VK/SK bytes after actual raw KeyGen's two sequential exports.
- Exact Sign/Verify traces, *accept* $\Rightarrow$ *tail*, and accepted hash-input binding expose actual caller control and hash arguments.
- Actual Sign's signature/siglen outputs preserve explicitly disjoint VK/SK/pre/message regions and retain KeyGen's external key prefix.
- Actual generated hashes are compared using only byte equality on the VK/pre/message read regions, instead of whole-memory equality.
- Executing actual raw Sign followed by cryptolab Verify corresponds exactly to sequential execution of the two traces, including results and final memory.

`raw_api_region_reaches_mu_zero_loss` converts these actual external-memory facts and Sign import into `sk_memory_prefix`. This path does not use the constructed `stores` of verified theorem 4.2.

6.2 Frozen security gate: product/replay for stored μ

The former **OBL-API-KEY-MEMORY-RAW** was decomposed in Week 5 because it was an unconditional statement stronger than the implementation's early-reject control flow. Since Week 6, the direct remaining obligation has narrowed to one semantic edge.

Open obligation 6.1 (**OBL-DIRECT-OBSERVED-MU**)**.** After sequential execution of the actual raw Sign trace and a successful cryptolab Verify trace, directly conclude

$$\mathrm{MuPrefix}_{32}(\mu_{S,\mathrm{trace}}, \mu_{V,\mathrm{trace}})$$

where the two terms are respectively `SignRawApiMuTrace . observed _ mu` and `VerifyCryptolabMuTrace . observed _ mu`. A justified product/replay rule is needed to transport the pRHL theorem for generated hash returns to the ghost fields of the two already completed traces.

verified theorem 4.9, verified theorem 4.10, and the actual sequential trace already provide frame, locality, control, and input binding. However, the signing continuation after the Sign hash differs from the challenge/compare continuation after the Verify hash, so one continuation cannot be discarded without the still-open signing losslessness result. Week 7 rejected additional observation wrappers, preconditions that repeat the goal, and SHAKE locality axioms, and froze **OBL-MU-PRODUCT-REPLAY** as an explicit **DEFERRED** paper boundary. Thus this lane does not record $\delta_{\mu,\mathrm{raw_api_accept}} = 0$, and no further wrappers are stacked on the same seam.

6.3 Recently closed gates: actual rANS core and production full HBZ

Weeks 7–11 successively closed the canonical signature prefix, HBZ leaves/tables, pure byte traces, actual suffix copy and harness control, and actual encoder successful-suffix refinement. Week 12 closed **OBL-RANS-DECODE-REFINEMENT** as exact-trace partial correctness. Week 13 composed these three results inside `Mode2RansActualHarness.run`, closing **OBL-RANS-CORE-INVERSE** as success-conditioned Hoare partial correctness. Week 14 composed actual full-HBZ prepare/apply with this core inverse and transported it exactly to the production SignaturePack/Unpack boundary, closing **OBL-SIG-HBZ-ENCODE-DECODE** at the same strength of partial correctness.

The premises of `actual_rans_encode_copy_decode_inverse` are only initial argument binding and `mode2_hbz_symbol_stream`. If the encoder actually fails, neither copy nor decoder runs, and the initial decoder array/state is retained. If the encoder succeeds, the full Week 12 exact-trace input is constructed from the actual copied segment, yielding

$$\begin{aligned} bad_{\mathrm{dec}} &= 0, & off_{\mathrm{dec}} &= \mathsf{encoded_size}, \\ decoded[0..1024) &= expected[0..1024), & decoded[1024..2048) &= decoded_0[1024..2048) \end{aligned}$$

The copied trace, pointwise decoder reads, decoder success, and output equality are not preconditions. `encoder_success_size_word_bridge` also derives non-wrapping W64 size subtraction from the actual success bounds, so no arbitrary integer-buffer witness is assumed.

`signature_pack_unpack_hbz_full_actual_exact` exactly matches the return tuples and `decoder_ran` of the production harness and focused full-HBZ harness. Its Hoare corollary `signature_pack_unpack_hbz_full_inverse_mode2` concludes zero-size failure or

nonzero-size success from canonical HBZ alone. The success branch gives (4≤size≤1024), (bad=0), 1024-coefficient recovery, coefficient/encoded tail frames, and the actual prepare-symbol trace. Neither encoder success nor the decoder result is a premise.

Verified theorem 6.1 (**OBL-RANS-ACTUAL-SUCCESS-WITNESS**)**.** Week 15's `actual_rans_encode_all_six_success` proves that, on the all-6 symbol stream produced by actual prepare, a terminating actual encoder returns $bad = 0$, $0 \le off \le 1020$, $4 \le size \le 1024$, and the exact trace/frame. `signature_pack_unpack_hbz_zero_success_mode2` lifts this to the production pack/unpack round trip for fixed all-zero HBZ. This fixed-input Hoare obligation is **PROVED** because success, size, and trace are not premises.

The Week 15 verdict was `GO-WITNESS`, not `GO-WITNESS-LOSSLESS`. Termination, losslessness, probability-1 success of the fixed-input execution, and success for general canonical inputs remain separate. The subsequent one-week minimum paper scope superseded the original (h)-codec recommendation, so **OBL-SIG-H-ENCODE-DECODE** is currently **DEFERRED**.

6.4 KeyGen gate: from frozen STOP-KG-NTT to current STOP-KG-AVEC-QJ

Week 16 composed actual `_kp_m23_matrix` and `_keypair_finalize_m23` in a transparent two-call harness. `actual_m23_matrix_finalize_semantic_snapshot` closes the actual intermediate `pre_bp`, NTT scratch, finalizer low/high semantics, and the snapshot mod-2q zero equality. This proves KG-2-style coefficient decomposition and the adjusted s_2 component of KG-4, but not the full paper KeyGen equation.

Boundary statement 6.1 (**STOP-KG-NTT** at paper freeze)**.** Connecting the full-NTT–pointwise-product–inverse-NTT result represented by `KeygenM23ArithmeticSpec.output_row` to the security model's row-wise polynomial product

$$\sum_{\text{col}} A_{\text{gen}}[\text{row}, \text{col}] s_{\text{gen}}[\text{col}]$$

required an odd-root orthogonality/full-NTT convolution theorem and an `Rq.poly`–security-list adapter. Neither was present at the frozen 82-target boundary, so that claim matrix does not establish faithful KG-1, KG-3, full KG-4, or $As = qj \pmod{2q}$.

Post-freeze work closed `rq_mul_coeff_foldr_to_bigi`, full-NTT spectral action, general/mode-2 row products, and the Rq–HAETAE coefficient-representation adapter. Subsequently, the actual mode-2 sampler's nonces 515/516, SHAKE128 framing, little-endian parsing, and rejection semantics were transported to paper a, and a separate paper qj correction was composed with the existing snapshot equality.

Boundary statement 6.2 (Current **STOP-KG-AVEC-QJ**)**.** The existing security model's `public_rounding_vector_seed` is a synthetic generator based on tag 401. It equals neither the actual sampler nor paper qj, and `security_named_qj_is_not_paper_qj` proves $401 \neq 64513$ at row 0 coefficient 0. The actual first nonce-515 coefficient for an all-zero raw seed is also 44985, so this identity cannot be assumed. **OBL-KG-SECURITY-EXPANDVECA-SEMANTICS**, replacing/refining the security model with actual ExpandVecA semantics, must precede promotion of KG claims.

Thus the frozen **OBL-MINCORE-KEYGEN** status remains **PARTIAL** for KG-2/finalization. The current auxiliary frontier also does not promote full KG-1–KG-4. The frozen MINCORE Sign verdict is `STOP-SIGN-CHAL-MODE2`. The frozen MINCORE Verify verdict is `STOP-VERIFY-MATRIX-CRT`, and restricted composition is **SPECIFIED**. Verify's pure NTT foundation was closed post-freeze, but the actual procedure leaves `verify_matrix_ntt_acc_mode2_cols4_correct` and `verify_crt_freeze_mode2_word_exact` remain.

6.5 Separate functional-correctness path

OBL-SIGN-OUTPUT-TAIL-REACH is a **PARTIAL** obligation stating that legitimate Sign outputs pass Verify's unpack/norm early-reject gates. The signature prefix inverse is closed, but the full suffix and subsequent gates remain open. This is neither the converse nor a consequence of verified theorem 4.7, which proves tail reachability for successful Verify executions. It requires the full pack/unpack inverse, norm, hint, response, and arithmetic correctness. **OBL-SIGN-VERIFY-CORRECTNESS**, combining this with full challenge equality, is currently **BLOCKED**. This functional path is not inserted as a hidden premise of the accepted-forgery security path.

6.6 Next gates: context and the full challenge transcript

The raw/internal and public-context paths actually differ in the implementation. Sign uses `_sf_prepare_pre_raw` and `_sf_mu_preptr`, while Verify selects the context branch using the most significant bit of `prelen`. The following must be shown.

1. $ctxlen \leq 255$ and correctness of length encoding;
2. Both sides absorb exactly $ctxlen$-byte$\|ctx$ in the same order;
3. Caller memory supplies the pointer/length premises for both procedures.

Next, **OBL-HIGH-LSB-PACK** must show equality of the packed bytes of HighBits_h, LSB produced by Sign and Verify. Composing this result with the existing `challenge_input_eq_components` gives equality of the full challenge input lists. Only then can the actual full call, including FIPS202 domain/padding/squeeze and the binary challenge sampler, be connected to a paper ROM query.

6.7 Subsequent functional and probabilistic obligations

Table 3: Main currently named unresolved obligations

Obligation	Nature	Required result
OBL-API-KEY-MEMORY-RAW-ACCEPT	API/relational	Direct stored μ_{32} equality for accepted Sign/Verify traces; frame/locality/control are already proved
OBL-DIRECT-OBSERVED-MU	API/relational	An `observed_mu` prefix conclusion in the actual sequential-trace post-state
OBL-MU-PRODUCT-REPLAY	Semantics; **DEFERRED**	A justified rule transporting the pRHL theorem for generated hash returns to completed-trace ghost fields
OBL-SIG-PREFIX-FRAME	Encoding	Preservation of the pack signature tail and challenge-output tail; the unpack low tail is proved
OBL-RANS-DECODE-REFINEMENT	Encoding; **PROVED**	Actual decoder $bad = 0$, exact consumed size, 1024-symbol equality, and output-tail frame under the exact actual trace, state fields, and mode-2 tables; termination is separate

Obligation	Nature	Required result
OBL-RANS-ACTUAL-SUCCESS-WITNESS	Encoding; **PROVED**	A terminating actual encoder returns $bad = 0$ and the exact size/trace/frame on fixed input consisting entirely of symbol 6; termination, losslessness, and phoare = 1 are separate
OBL-RANS-CORE-INVERSE	Encoding; **PROVED**	Success-conditioned partial correctness preserving the actual encoder failure branch and, on success, transporting the actual copied segment to the decoder's exact-trace input to obtain $bad = 0$, exact consumption, 1024-symbol recovery, and the tail frame; termination/success reachability are separate
OBL-SIG-HBZ-ENCODE-DECODE	Encoding; **PROVED**	Success-conditioned partial correctness preserving zero-size failure in the production full-HBZ wrapper and supplying size bounds, $bad = 0$, 1024-coefficient recovery, two tail frames, and the prepare-symbol trace on nonzero-size success; failure exclusion for fixed zero input is proved, while termination/general-input success are separate
OBL-SIG-H-ENCODE-DECODE	Encoding; **DEFERRED**	Actual hint-codec inverse deferred in favor of the Week 16 minimum KeyGen scope
OBL-MINCORE-KEYGEN	Functional correctness; **PARTIAL**; `STOP-KG-NTT`	Frozen 82-target claim through the actual snapshot/finalizer and final `output_row` rewrite. Post-freeze NTT/Rq auxiliary theorems advanced this historical stopping point, but promote neither the frozen claim itself nor the full KG equation
OBL-KG-SECURITY-EXPANDVECA-SEMANTICS	Functional correctness; **BLOCKED**; `STOP-KG-AVEC-QJ`	Requires an actual-array representation theorem replacing/refining synthetic `public_rounding_vector_seed` with actual SHAKE128/LE16/rejection and nonce 515/516 semantics while retaining paper qj as a separate correction
OBL-MINCORE-SIGN	Functional correctness; **PARTIAL**; `STOP-SIGN-CHAL-MODE2`	Only direct calls to the actual round-challenge, z-check, and hint helpers and accepted-branch control compile; full challenge semantics and S-1–S-7 headline claims are unproved

Obligation	Nature	Required result
OBL-MINCORE-VERIFY	Functional correctness; **PARTIAL**; `STOP-VERIFY-MATRIX-CRT`	Actual helper order, V-1/V-2/V-5/V-6 machine-word equations, the W64 norm gate, and tail/mismatch word expressions compile; the post-freeze pure NTT foundation is proved, but actual 2-by-4 matrix accumulation, the CRT/freeze semantic bridge, and challenge equality are unproved
OBL-MINCORE-COMPOSITION	Functional correctness; **SPECIFIED**	Restricted Sign–Verify composition at the decoded-object boundary remains unproved
OBL-SIG-SUFFIX-METADATA, **OBL-SIG-SUFFIX-PADDING**	Encoding	Size-delta round trip and 1474-byte canonical zero padding
OBL-SIG-FULL-CODEC-MODE2	Encoding	Actual pack/unpack inverse combining the prefix and full suffix, a successful bad flag, and canonical parsing
OBL-SIGN-OUTPUT-TAIL-REACH	Functional correctness	Full pack/unpack and norm/hint gate passage for legitimate Sign outputs
OBL-SIGN-VERIFY-CORRECTNESS	Functional correctness	Acceptance of legitimate signatures, including response, hint, arithmetic equations, and the full challenge
OBL-KG-TERMINATION, **OBL-KG-DIST**	Probability; termination	Retry-loop losslessness or termination, actual KeyGen output distribution, and the effects of singular rejection
OBL-FFT-TAIL, TieMin	Numerical analysis	Bad-event probability for fixed-point FFT errors and differences arising from the tied-minimum policy
OBL-SIGN-ACCEPTED-DIST	Probability distributions	The hyperball sampler, rejection, and accepted outputs agree with the paper distribution or incur explicit statistical loss
OBL-VERIFY-EVENT	Algebra; programs	Jasmin's final accept bit is equivalent to the paper's norm, hint, equation, and challenge conditions
OBL-CHALLENGE	Combinatorics; probability	Challenge support, cardinality, point probability, and min-entropy
OBL-FORK	Cryptographic reduction	Extraction of the exact BST-MSIS instance from two fresh transcripts
OBL-SIGROM-ADAPTER	Game-based security	Instantiation of the byte-level public API as the exact `Sig_ROM.Scheme`, with query accounting

6.8 Dependency-based work order

The following order best matches the current code.

1. Retain the product/replay path as the current paper boundary and add no more observation wrappers at the same caller seam.
2. Preserve the Week 14 production wrapper and Week 15 fixed all-6 success theorem without reproving the rANS loop invariants. Leave fixed-input termination/losslessness as separate obligations.
3. Retain `STOP-KG-NTT` in the frozen claim matrix as a historical boundary. Reuse the NTT/Rq–HAETAE theorems closed post-freeze, without retroactively recording them as claims of the frozen PDF.
4. Resolve the current `STOP-KG-AVEC-QJ` first. Replace/refine synthetic `public_rounding_vector_seed` with actual ExpandVecA semantics, separate it from the paper qj correction, and prove `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS`.
5. Connect security a, the actual array, and the Rq/HAETAE row product, then transport the mode-2 width-4/mod-q representation to the paper's augmented width-6/mod-$2q$ matrix, secret, and error packaging. Before that, do not claim faithful KG-1/KG-3/full KG-4.
6. Retain `STOP-SIGN-CHAL-MODE2`. Do not claim S-1–S-7 without the full highbits/LSB/μ-to-SampleInBall leaf, application of the actual Sign row product, and response/hint representation obligations.
7. For Verify, close `verify_matrix_ntt_acc_mode2_cols4_correct`, which applies the already closed pure NTT foundation to actual 2-by-4 accumulation, and `verify_crt_freeze_mode2_word_exact`; compose the combined matrix theorem, then close the downstream challenge leaf.
8. Keep the (h) codec deferred for now. When its scope is reopened, compose size metadata, zero padding, and pack/challenge tail frames to close the full 1474-byte codec and canonical parsing.
9. Combine norm/hint, response, and arithmetic correctness to close **OBL-SIGN-OUTPUT-TAIL-REACH** and legitimate Sign/Verify correctness.
10. In the separate security path, compose public-context μ, highbits/LSB, and the full challenge call.
11. Quantify KeyGen/Sign distribution and termination losses, then combine the byte-faithful ROM scheme with the paper's distribution/forking obligations.

This order advances one step at a time from the nearest already-compiling seam toward the outer boundary, rather than declaring broad security statements first and filling their underlying premises afterward.

7 Trust boundaries and reproduction

7.1 What is trusted

The trusted computing base (TCB) of this proof development includes the following.

- The EasyCrypt kernel/type checker and imported standard libraries;
- Why3 and the invoked SMT provers;
- The Jasmin parser/compiler and `jasmin2ec` translator;

- Hash-pinned Jasmin sources, generated extractions, and reused EasyCrypt artifacts;
- Host tools performing source hashing and LaTeX builds.

Acceptance of the source by `jasmin2ec` and compilation of the generated EasyCrypt do not prove the translator's semantic preservation. Unless there is a separate verified-translator theorem, the translator itself remains in the TCB. Likewise, memory safety and SCT/constant-time properties do not follow from EUF-CMA and form separate tracks.

7.2 Proof verification pipeline

The commands in this historical proof pipeline require the complete earlier tree restored as described in the archive README. Run them from that restored repository root.

```
./haetae-topdown-easycrypt/scripts/verify-all.sh
./haetae-topdown-easycrypt/scripts/check-paper-freeze.sh
python3 haetae-topdown-easycrypt/post-freeze/check-kg-actual-avec-qj-trace.py
```

The first command checks the frozen 82-target aggregate, the second explicitly checks the paper-freeze scope, and the third checks an independent deterministic witness of the latest post-freeze mismatch.

The script checks the following in order.

1. SHA-256 drift in the pinned sources and read-only proof tree;
2. Regeneration and hash agreement of focused extractions for the packer, signature codec, HBZ prepare/apply/full wrappers, API key-copy helpers, raw API callers, raw μ, and transcripts; Week 7 pack/unpack and Week 8 focused HBZ procedure-body identity; and identity of generated targets reused by the Week 9 actual rANS harness;
3. Deterministic regeneration of the actual 512-word HBZ symbol certificate, generator/output hash agreement, and kernel rechecking of the generated EasyCrypt certificate itself;
4. In addition to Week 9 rANS theorem surfaces, overclaim checks for Week 10 array/list, word-step, inner-progress, actual inner-loop, serialization, and success-size theorem surfaces; Week 11 tail-aware invariants, generated word steps, final stores, and actual encoder trace closure; Week 12 decoder cursor/table/replay invariants and actual trace refinement; Week 13 copy traces, pointwise inputs, W64 size adapters, and the actual core inverse; Week 14 full/focused internal equivalence, full encoder/decoder, direct harness, and production-boundary lift; Week 15 fixed all-six budget, failure causes, and production success lift; and frozen-surface checks for Week 16 actual M23 matrix/finalizer snapshots, the final NTT representation rewrite, direct Sign/Verify core control, and `STOP-VERIFY-MATRIX-CRT`;
5. Fresh `-no-eco` compilation of the 82 currently manifested authored EasyCrypt targets;
6. Absence of `admit`, `sorry`, authored `axioms`, and debug/temporary declarations, and manifest completeness;
7. Fresh compilation of selected upstream baselines;
8. Builds of the existing sprint LaTeX research notes without reference/citation errors;
9. A paper-freeze audit checking that the frozen PDF, 82 targets, claim matrix, and exact blockers in `manifests/paper-artifacts.md` remain unchanged.

Extraction hashes detect generated-artifact drift but do not replace justification of translation semantics. Run summaries are also overwritten on every run, so publication snapshots must retain complete logs including the final RESULT PASS line. The terminal marker of the 76-target completion snapshot is preserved in logs/verify-all-before-kg-ntt-mul.log as RESULT PASS authored-targets=76 cache=-no-eco. The 77-target completion marker before the Sign changes is preserved in logs/verify-all-before-week16-sign.log, and WEEK16_SIGN_REPORT.md describes the 78th target's direct actual-call control and stop verdict. The 78/78 completion log before the Verify changes is preserved in logs/verify-all-before-week16-verify.log, with hash cf8056712327dc8211cf93ae427ac505
3e8a9d2366747f171392468ac3ff0d75. WEEK16_VERIFY_REPORT.md describes STOP-VERIFY-MATRIX-CRT and the remaining blockers at the canonical decoded (x, v, h, c) boundary. The 82/82 aggregate log immediately before paper freeze is preserved in
logs/verify-all-before-paper-freeze.log, with SHA-256
4cd64e5a656be82710bca1410c4d194
03a3c661d6b91b0319a0ea8f7c91646da.
logs/verify-all-summary.txt, tracked at baseline commit e87bcc3, retains the terminal RESULT PASS
authored-targets=82
cache=-no-eco, including a passing paper-freeze audit, and SHA-256
08ef9639dc73d56dba42d02999d07897
d29bc8e60aa100a648e9437fd64387ad. The frozen paper PDF hash is
be935948028829556951863b44ceb2e6
c5b2037820991b3a43a4b461b036e53d.

Boundary statement 7.1 (82-target paper freeze and separate post-freeze auxiliary layer)**.** The Week 14 completion run on 2026-08-10 closed 72 targets by adding the following five theories to the Week 13 67-target baseline.

- Mode2HbzInternalBoundaries.ec;
- Mode2HbzFullEncodeTrace.ec, Mode2HbzFullDecodeInverse.ec;
- Mode2HbzFullActualInverse.ec, Mode2HbzSignatureBoundaryLift.ec.

Week 15 added Mode2RansAllSixBudget.ec and Mode2RansActualSuccessWitness.ec, expanding to 74 targets; Week 16 added Mode2KeygenSnapshotAlgebra . ec and Mode2KeygenCoreEquation . ec, expanding to 76 targets. The 77th target at paper freeze, added by the KG-NTT-MUL audit, Mode2KeygenNttMulBridge . ec, individually fresh-compiled the final output_row representation rewrite and the two active-row consequences for the actual two-call harness. This frozen version assumes neither odd-root convolution nor a security-list adapter; its verdict was STOP-KG-NTT. logs / week14-hbz-full-surface-scan . log checks theorem surfaces for direct production calls, preconditions, failure/success branches, and frame conclusions. The Week 13 independent verification record separately confirms that the underlying actual rANS core inverse has no circular premises. logs / week15-rans-success-surface-scan . log checks that the fixed-input theorem does not assume success, trace, capacity, or losslessness, and logs/week16-keygen-core-surface-scan.log checks the two actual calls, non-circular preconditions, that the NTT boundary does not hide the desired product, and the STOP-KG-NTT boundary. logs / week16-sign-core-surface-scan . log checks the three actual helper calls, non-circular preconditions, and stop boundary. Intermediate PASS lines in logs / verify-all-summary . txt, generated during execution, are not completion evidence; only a terminal RESULT together with a fixed hash is used as current aggregate evidence. The 78/78 completion log before the Verify changes is preserved in logs / verify-all-before-week16-verify . log, and the 82/82 aggregate immediately before paper freeze is preserved in logs/verify-all-before-paper-freeze.log.

Post-freeze NTTFullSpectralAction.ec, NTTMode2RowSpecializations.ec, RqHAETAEBridge.ec were fresh-compiled in the external NTT tree, and the strengthened

Mode2KeygenNttMulBridge.ec is recompiled within the current 82-target aggregate. The three post-freeze/KgActualAvecQj*.ec targets were each compiled separately with -no-eco, without adding them to the frozen manifest. Thus 82 is not a count of all post-freeze auxiliary theorems; it means that the frozen proof-target surface has been maintained.

7.3 Building this document

This guide is a XeLaTeX document independent of the existing sprint notes. To build the English edition, run the following in this archived guide's directory, using the fonts listed in its README:

```
latexmk -xelatex -interaction=nonstopmode -halt-on-error main.tex
```

This builds the translated PDF only. The original build script below requires the complete earlier tree. From that restored repository root, run

```
sh haetae-topdown-easycrypt/theory-guide/build.sh
```

to generate haetae-topdown-easycrypt/theory-guide/main.pdf. The script treats fatal errors and undefined references/citations as failures.

The tool versions observed when writing this document were Jasmin/jasmin2ec 2026.03.0, EasyCrypt OPAM switch easycrypt-5.2, and Why3 1.8.0; configured provers included Alt-Ergo 2.6.0, CVC4 1.8.0, CVC5 1.2.1, and Z3 4.12.6. For exact reproduction results, execution logs take precedence over version strings.

Completion criteria

The following criteria concern updates to the historical research baseline; the English edition does not claim a new proof replay. Compilation of this guide and compilation of EasyCrypt theorems are different forms of evidence. Before distributing the document, both build.sh and the full verify-all.sh must succeed. The current frozen proof baseline is the terminal PASS of the 82/82 aggregate and paper-freeze audit. Post-freeze claims additionally require separate checks of fresh compilation of the three actual-avec/paper-qj targets, the Rq/NTT auxiliary targets, and the deterministic trace. Intermediate PASS lines in a summary being overwritten are not completion evidence. The separate build of this guide must also be checked.

A Theorem and source index

Table 4: Stable index for locating the main declarations in the text

Declaration or claim	File	What to check when reading
FC_KeyGen, FC_Sign, FC_Verify	easycrypt/specs/TopLevelGoals.ec	Top-level specification equalities for implementation/paper distributions and the verification function
ImplementationSecurityComposition	easycrypt/specs/TopLevelGoals.ec	Abstract loss budget of equation (1)
implementation_security_obligations	easycrypt/security/ImplementationSecurityGoals.ec	Conjunction of explicitly exposed implementation-security obligations

Declaration or claim	File	What to check when reading
`cryptographic_bounds_well_formed`	`easycrypt/security/SecurityAssumptions.ec`	MLWE, BST-MSIS, and ROM assumption interfaces
`vk_prefix_eq`	`easycrypt/refinement/keygen/PackedKeyPrefix.ec`	Array-level definition of definition 3.1
`pack_sk_m23_mode2_vk_prefix`	`easycrypt/refinement/keygen/ExtractedPackedKeyPrefix.ec`	992-byte prefix partial correctness of the actual generated packer
`keypair_internal_mode2_return_prefix`	`easycrypt/refinement/keygen/ExtractedPackedKeyPrefix.ec`	Lift to actual internal mode-2 KeyGen return values
`imported_mode2_full_first_attempt_equiv`	`easycrypt/refinement/keygen/ExistingFirstAttemptAdapter.ec`	Import-only bridge that does not strengthen the existing first-attempt theorem's premises
`snapshot_residue_exact_low_high`, `snapshot_expression_from_product_congruent_mod_2q`	`easycrypt/refinement/keygen/Mode2KeygenSnapshotAlgebra.ec`	Week 16 pure integer layer providing exact low/high decomposition of the actual finalizer residue and mod-$2q$ congruence of the snapshot expression
`actual_m23_matrix_finalize_snapshot`, `actual_m23_matrix_finalize_semantic_snapshot`	`easycrypt/refinement/keygen/Mode2KeygenCoreEquation.ec`	Week 16 Hoare theorem directly composing actual `_kp_m23_matrix` and `_keypair_finalize_m23` to expose `pre_bp`, exact finalizer semantics, low/high, and frames
`output_row_from_mode2_ntt_words`, `output_row_repr_from_mode2_ntt_words`	`easycrypt/refinement/keygen/Mode2KeygenNttMulBridge.ec`	The 77th manifested target, expanding `output_row` into the existing full-invNTT/pointwise-word expression and transporting existing slice representations. The two active-row consequences of the actual two-call snapshot are also individually compiled. This is the paper-freeze `STOP-KG-NTT` surface; the current file also reuses the post-freeze row-product/Rq–HAETAE results below
`rq_mul_coeff_foldr_to_bigi`, `full_ntt_montgomery_spectral_action`, `full_ntt_montgomery_row_product`	`haetae-ntt-verify/easycrypt/NTTFullSpectralAction.ec`	Post-freeze pure NTT foundation closing checked negacyclic coefficient sums, Montgomery factors, the 256-term odd-root kernel, and full-NTT spectral action; does not use legacy `Rq.ntt`
`keygen_mode2_row_product_from_reprs`, `verify_mode2_row_product_from_reprs`	`haetae-ntt-verify/easycrypt/NTTMode2RowSpecializations.ec`	Specializes the general row product to KeyGen 2-by-3 and Verify 2-by-4 mathematical representations; not the actual Verify procedure leaf itself
`rq_poly_add_repr`, `rq_poly_mul_repr`, `rq_poly_dot3_repr`	`haetae-ntt-verify/easycrypt/RqHAETAEBridge.ec`	Post-freeze representation adapter transporting Rq addition, multiplication, and three-term dot products to HAETAE list-polynomial operations under a well-formed coefficient relation

Declaration or claim	File	What to check when reading
keygen_mode2_row_product, mode2_row_product_haetae_dot	easycrypt/refinement/keygen/Mode2KeygenNttMulBridge.ec	Native mode-2 Rq row product and a bridge to the HAETAE three-term dot product under six coefficient-representation premises; concludes neither KG-3/KG-4 nor the final equation
mode2_uniform_stream_paper_a_flat, paper_qj_mode2_cases	post-freeze/KgActualAvecQjSemantics.ec	Deterministic partial correctness transporting nonce 515/516, SHAKE128 framing, and LE16/$<$ 64513 rejection from actual avec to paper a, while separating the basis position of paper qj
checked_mode2_parent_m23_finalize_actual_avec_qj	post-freeze/KgActualAvecQjComposition.ec	Exposes both the paper a interpretation and snapshot $+qj$ equation for terminating executions of the checked sampler, actual matrix, and finalizer; claims neither the KG equation, distribution, nor termination
security_named_qj_is_not_paper_qj	post-freeze/KgActualAvecQjBlocker.ec	Directly proves the object mismatch between security-side synthetic row 0 coefficient 401 and paper $qj[0][0] = 64513$, justifying STOP-KG-AVEC-QJ
keygen_prefix_memory_relation_is_constructible	easycrypt/support/Mode2KeyMemoryBridge.ec	Historical local witness constructing a memory prefix from a stored VK
keygen_export_vk_mode2_prefix, keygen_export_sk_mode2_prefix	easycrypt/refinement/composition/ApiKeyMemoryBridge.ec	992/1408-byte memory-prefix partial correctness of the actual KeyGen exporters
sign_import_mode2_sk_prefix, verify_import_mode2_vk_prefix	easycrypt/refinement/composition/ApiKeyMemoryBridge.ec	Local 992-byte prefix partial correctness of the actual Sign/Verify importers
exported_mode2_regions_agree, imported_sign_sk_reaches_mu_memory	easycrypt/refinement/composition/ApiKeyMemoryBridge.ec	Pure adapter from export/import prefixes to raw μ memory premises; caller composition is separate
canonical_ui64_address_roundtrip, mode2_vk_region_bridge	easycrypt/refinement/composition/RawApiAddressBridge.ec	Canonical round trips and region binding for raw integer addresses and W64 addresses
keypair_raw_api_exact_export_trace	easycrypt/refinement/composition/RawApiKeygenExportComposition.ec	Exact memory/result equivalence between actual raw KeyGen and the export trace containing two copies
keypair_raw_api_exports_matching_prefixes	easycrypt/refinement/composition/RawApiKeygenSequentialExport.ec	External VK/SK prefix equality after actual raw KeyGen sequential exports
sign_raw_api_exact_mu_trace, sign_raw_api_trace_binds_hash_inputs	easycrypt/refinement/composition/RawApiCallerMuTrace.ec	Exact relation between actual raw Sign and its trace, with hash-input observations
raw_api_region_reaches_mu_zero_loss	easycrypt/refinement/composition/RawApiMuReachability.ec	Connects the external prefix and Sign import to sk_memory_prefix without a constructed store
verify_raw_api_exact_mu_trace, verify_cryptolab_exact_mu_trace	easycrypt/refinement/composition/RawApiVerifyMuTrace.ec	Traces preserving actual Verify results and memory in every early-reject/tail branch

Declaration or claim	File	What to check when reading
verify_raw_api_actual_accept_binds_hash_inputs	easycrypt/refinement/composition/RawApiVerifyAcceptTrace.ec	Tail reachability and VK/pre/message hash-input binding for actual successful Verify
raw_api_accepting_execution_hash_mu_zero_loss	easycrypt/refinement/composition/RawApiAcceptedMuComposition.ec	Instantiates the actual generated hash-call μ prefix from accepted-trace facts
raw_api_accepting_execution_generated_challenge_suffix_zero_loss	easycrypt/refinement/composition/RawApiAcceptedMuComposition.ec	Position-64 generated suffix-helper result equality under accepted-trace premises
sign_output_copy_exact_and_frames_reused, sign_raw_api_frames_reused_regions	easycrypt/refinement/composition/RawApiSignOutputFrame.ec	Actual signature/siglen outputs and the actual raw Sign caller preserve disjoint VK/SK/pre/message regions
sign_verify_generated_raw_mu_prefix_regionwise	easycrypt/refinement/composition/RegionLocalMuEquivalence.ec, RegionLocalMuTop.ec	Compares actual generated hashes by VK/pre/message read region without whole-memory equality
raw_sign_then_verify_actual_exact_trace, sign_then_verify_accept_binds_observed_inputs	easycrypt/refinement/composition/RawApiDirectObservedMu.ec	Result/memory equivalence of the actual Sign-to-Verify sequence and trace, with accepted-descriptor postconditions; no stored μ equality
sign_verify_raw_mu_top_wrappers_prefix	easycrypt/refinement/composition/ExtractedMuHashPrefix.ec	Wrapper-level μ_{32} prefix of the generated helper chain
sf_mu_rawpre_refines_raw_top	easycrypt/refinement/composition/ExactMuTopControl.ec	Exact relation between the actual Sign raw top and wrapper
verify_hash_mu_raw_refines_raw_top	easycrypt/refinement/composition/ExactMuTopControl.ec	Exact relation between the actual Verify raw branch and wrapper
sign_verify_generated_raw_mu_prefix	easycrypt/refinement/composition/ExactMuTopControl.ec	μ_{32} prefix between actual generated Sign/Verify procedures
sign_verify_mu32_absorb_from_pos64	easycrypt/refinement/composition/ExtractedChallengeAbsorb.ec	Equality of actual μ_{32} absorb-helper results at position 64
generated_raw_mu_preconditions_from_keygen_prefix	easycrypt/refinement/composition/ExactMode2RawMuComposition.ec	Concrete witness of raw-theorem premises from the KeyGen prefix
generated_raw_mu_to_challenge_suffix_zero_loss	easycrypt/refinement/composition/ExactMode2RawMuComposition.ec	Zero-loss composition theorem for the current local raw generated-top seam
challenge_input_eq_components	easycrypt/refinement/composition/TranscriptBytes.ec	Congruence giving equality of the entire list transcript when highbits, LSB, and μ respectively agree
canonical_challenge, canonical_signed_low, prefix_codec_preconditions_satisfiable	easycrypt/refinement/sign/Mode2SignaturePrefixCodec.ec	Canonical challenge/signed-low premises of the prefix round trip and an all-zero non-vacuity witness

Declaration or claim	File	What to check when reading
pack_sig_prefix_mode2_layout, unpack_sig_prefix_mode2_layout	easycrypt/refinement/sign/Mode2SignaturePrefixPack.ec, Mode2SignaturePrefixUnpack.ec	32-byte challenge and 1024-byte low layouts of actual generated prefix pack/unpack, with the unpack low-tail frame
pack_unpack_sig_prefix_mode2_roundtrip	easycrypt/refinement/sign/Mode2SignaturePrefixRoundTrip.ec	Partial correctness of the canonical mode-2 prefix round trip through the two actual generated procedures
canonical_hbz_mode2, canonical_hbz_mode2_satisfiable	easycrypt/refinement/sign/Mode2HbzCodecSpec.ec	Mode-2 1024-coefficient range, leaf prefix/frame predicates, and all-zero canonical witness
encode_hb_z1_prepare_mode2_correct, decode_hb_z1_apply_mode2_correct	easycrypt/refinement/sign/Mode2HbzPrepare.ec, Mode2HbzApply.ec	Exact symbol mapping, bad word, coefficient recovery, and bounded tail frame of the actual prepare/apply leaves
encode_prepare_decode_apply_mode2_inverse	easycrypt/refinement/sign/Mode2HbzLeafRoundTrip.ec	Recovery of the first 1024 HBZ coefficients by sequential composition of the two actual leaf wrappers
pack_target_encode_hb_z1_full_exact_focused, unpack_target_decode_hb_z1_full_exact_focused	easycrypt/refinement/sign/Mode2HbzActualBoundary.ec	Focused/full and signature pack/unpack extraction paths expose the same actual HBZ full procedure
mode2_hbz_table_certificate, hbz_fast_step_matches_math	easycrypt/refinement/sign/Mode2HbzTableCertificate.ec	13-symbol interval/table fields and reciprocal quotient/overflow certificates
actual_mode2_hbz_tables_certified	easycrypt/refinement/sign/Mode2HbzSymbolWordsGenerated.ec	Regenerable theorem that literal actual esyms/dsyms/symbol arrays satisfy the table certificate
pure_rans_step_inverse, hbz_fast_step_decode_inverse	easycrypt/refinement/sign/Mode2RansCore.ec	Pure quotient/slot inverse and concrete reciprocal fast-step inverse; excludes normalization bytes and actual loops
renorm_bytes_readback, encode_trace_state_bounds, valid_rans_trace_canonical	easycrypt/refinement/sign/Mode2RansByteStack.ec	Theorems for 0/1/2-byte normalization, *xs*/*cuts*/*bytes* traces, little-endian state, and canonical state bounds
encoder_two_byte_decoder_order, cursor_two_decrements_no_underflow	easycrypt/refinement/sign/Mode2RansNormalization.ec	Integer semantics of actual W32 shift/truncate and decoder shift/or operations, with W64 backward-cursor safety
copy_encoded_suffix_correct	easycrypt/refinement/sign/Mode2RansSuffixCopy.ec	Pointwise slice equality and unused-tail frame of actual `__copy_encoded_suffix`
actual_rans_encode_mode2_control, actual_rans_decode_mode2_control	easycrypt/refinement/sign/Mode2RansEncodeRefinement.ec, Mode2RansDecodeRefinement.ec	Concrete mode-2 inputs/control of the actual generated loops and $bad \in \{0, 1\}$. Separately from these control theorems, encoder trace refinement was closed in Week 11 and decoder trace refinement in Week 12

Declaration or claim	File	What to check when reading
actual_rans_harness_ branches_on_encoder_ result	easycrypt/refinement/sign/ Mode2RansActualInverse.ec	Actual encoder–copy–decoder calls, failure/success branching on actual results, and decoder input-field initialization; the existing control theorem itself is not an inverse, and the Week 13 semantic theorem is listed separately below
symbol_list_of_array, symbol_suffix, encode_trace_suffix_ extension	easycrypt/refinement/sign/ Mode2RansArrayListBridge.ec	Connects the first 1024 symbols of the actual array to a list and its suffixes, and proves segment/frame operations and a one-symbol trace recurrence
actual_mode2_encoder_ word_step_correct	easycrypt/refinement/sign/ Mode2RansEncoderWordStep.ec	Word-level theorem equating the actual mode-2 tables, W32/W64 reciprocal products, and shift/bias words with the pure fast encoder step
generated_mode2_encoder_ word_step_unfold, generated_outer_word_ update_matches	easycrypt/refinement/sign/ Mode2RansEncoderGenerated WordStep.ec	Verified Week 11 bridge connecting generated table-field loads, reciprocal high products, shift ladders, complements, biases, and actual outer word updates to the existing fast step
normalization_len_cases, inner_progress_segment_ step	easycrypt/refinement/sign/ Mode2RansEncoder InnerProgress.ec	Pure normalization lengths 0/1/2, repeated division, exact low-byte suffixes, and one-step segment preservation
encoder_inner_phase_inv, encoder_inner_segment_ inv, encoder_inner_segment_ step	easycrypt/refinement/sign/ Mode2RansEncoderTrace.ec	Conservative 0..4 phases, byte-segment and prefix-frame predicates, and actual store steps for the actual generated inner loop
actual_w32_le_bytes_ serialize, actual_encoder_final_ state_ serialization	easycrypt/refinement/sign/ Mode2RansEncoder Serialization.ec	Little-endian serialization by four actual pattern set8 stores, outer/prefix frames, and success-size arithmetic leaf theorems
actual_store_w32_le_ prepend_trace_bytes	easycrypt/refinement/sign/ Mode2RansEncoder SerializationComposition.ec	Composition theorem prepending the serialized final state to the pure tail and preserving segment/frame relations to the initial array; used with the four actual stores in the Week 11 finalization theorem
actual_rans_encode_inner_ no_underflow, actual_rans_encode_ success_ size_bound	easycrypt/refinement/sign/ Mode2RansEncoder ActualInner.ec	Inner byte/frame invariants and failure-or-safe-offset postconditions of actual generated _rans_encode; direct Hoare partial correctness for successful sizes 4..1024
encoder_inner_tail_inv, encoder_outer_tail_ advance_success	easycrypt/refinement/sign/ Mode2RansEncoderTail Invariant.ec	Verified Week 11 layer providing initialization of inner/outer invariants carrying the pure tail, actual inner progress, exact 0/1/2-byte termination, and a one-symbol transition
encoder_outer_tail_ finalize_success	easycrypt/refinement/sign/ Mode2RansEncoder Finalization.ec	Finalization theorem combining pure final-state serialization with the accumulated normalization suffix and deriving exact length and the prefix frame

Declaration or claim	File	What to check when reading
generated_encoder_outer_finalize_success	easycrypt/refinement/sign/Mode2RansEncoderGeneratedFinalization.ec	Verified bridge connecting the generated offset word and four actual little-endian stores to the pure finalization theorem
encoder_trace_post, encoder_generated_success_outer_post, actual_rans_encode_trace_closure	easycrypt/refinement/sign/Mode2RansEncoderActualTraceClosure.ec	Core Week 11 Hoare theorem directly opening actual RansEncodeTarget.M._rans_encode to prove failure or, on success, the exact full suffix, length, offset, and frame
actual_rans_encode_failure_trace_cause	easycrypt/refinement/sign/Mode2RansEncoderActualTraceClosure.ec	Week 15 failure-cause theorem deriving that normalization trace length exceeds 1020 in a terminating execution where the actual encoder returns 0
actual_rans_encode_trace_refinement	easycrypt/refinement/sign/Mode2RansEncoderOuterRefinement.ec	Success-conditioned partial-correctness corollary expanding the core closure theorem into the actual return tuple and trace_bytes expression
decoder_state_at, decoder_cursor, decoder_cursor_final	easycrypt/refinement/sign/Mode2RansDecoderCursor.ec, Mode2RansDecoderCursorSteps.ec	Week 12 boundary theorems for decoder states, segments, initial/one-step/final cursors of pure traces, and each segment read preceding the total size
actual_mode2_decoder_word_update_correct, generated_decoder_symbol_expected	easycrypt/refinement/sign/Mode2RansDecoderWordStep.ec, Mode2RansDecoderActualWord.ec, Mode2RansDecoderGeneratedStep.ec	Connects actual symbol_words/dsyms_words lookups and W32/W64 updates to the mathematical decoder word step and expected symbol
decoder_replay_guard_exact, decoder_inner_trace_step, decoder_outer_trace_exit_components	easycrypt/refinement/sign/Mode2RansDecoderNormalization.ec, Mode2RansDecoderActualTrace.ec	0/1/2-byte partial replay, actual inner/outer loop invariants, exact consumption cursor, $x = 2^{23}$, output prefix, and tail frame
actual_rans_decode_trace_refinement	easycrypt/refinement/sign/Mode2RansDecoderTopHoare.ec	Week 12 Hoare partial correctness directly opening actual generated RansDecodeTarget.M._rans_decode to prove $bad = 0$, exact consumed size, 1024-symbol recovery, and output-tail preservation on exact-trace inputs
copied_suffix_is_exact_trace, segment_matches_implies_exact_decoder_segment_input, actual_decoder_input_from_configured_trace, encoder_success_size_word_bridge	easycrypt/refinement/sign/Mode2RansCoreCompositionBridge.ec	Week 13 adapter constructing global/pointwise exact-trace decoder inputs from the actual encoder segment and copied slice, and connecting actual state stores with non-wrapping W64 sizes

Declaration or claim	File	What to check when reading
actual_rans_encode_copy_decode_inverse	easycrypt/refinement/sign/Mode2RansCoreActualInverse.ec	Week 13 success-conditioned Hoare partial correctness composing, inside Mode2RansActualHarness.run, decoder non-execution/initial-state preservation on actual encoder failure and actual copy, decoder $bad = 0$, exact consumption, 1024-symbol recovery, and tail frame on success
full_encode_prepare_mode2_correct, full_rans_encode_trace_refinement, full_rans_decode_trace_refinement, full_decode_apply_mode2_correct	easycrypt/refinement/sign/Mode2HbzInternalBoundaries.ec	Exact equivalence of full-wrapper extractions and focused prepare/rANS/apply extractions, with canonical coefficient–symbol stream and decoder-prefix–apply-premise adapters
actual_encode_hb_z1_full_mode2_trace, actual_decode_hb_z1_full_mode2_inverse	easycrypt/refinement/sign/Mode2HbzFullEncodeTrace.ec, Mode2HbzFullDecodeInverse.ec	Directly proves the actual full encoder's failure/exact-trace branches and the actual full decoder's $bad = 0$, 1024-coefficient recovery, and coefficient tail frame, respectively
actual_hbz_full_encode_decode_inverse_mode2	easycrypt/refinement/sign/Mode2HbzFullActualInverse.ec	Composes zero-size failure or the nonzero-size success inverse and two tail frames from canonical HBZ alone in focused HbzFullActualHarness.run
signature_pack_unpack_hbz_full_actual_exact, signature_pack_unpack_hbz_full_inverse_mode2	easycrypt/refinement/sign/Mode2HbzSignatureBoundaryLift.ec	Exact equivalence of the production SignaturePack/Unpack harness and focused full-HBZ harness, and Week 14 Hoare partial correctness with the same failure/success contract
all_six_normalization_budget, all_six_trace_fits_mode2	easycrypt/refinement/sign/Mode2RansAllSixBudget.ec	Week 15 budget showing that zero HBZ prepares an all-six symbol stream, the first four symbols emit no bytes, and the rest emit at most one byte each, giving normalization length ≤ 1020 and total trace length 4..1024
actual_rans_encode_all_six_success, signature_pack_unpack_hbz_zero_success_mode2	easycrypt/refinement/sign/Mode2RansActualSuccessWitness.ec	Hoare partial correctness excluding failure on fixed zero input and transporting success postconditions through actual rANS, full-HBZ, and production pack/unpack; not a termination or losslessness theorem
CLAIM_LEDGER	CLAIM_LEDGER.md	Operational source of truth for paper-freeze claim statuses; separate reports supplement post-freeze auxiliary results
THEOREM_GRAPH	THEOREM_GRAPH.md	Dependencies from frozen parent security goals to Week 16 stop seams; post-freeze deltas are not retroactively incorporated
PRODUCT_REPLAY_DECISION	PRODUCT_REPLAY_DECISION.md	Semantic rationale for freezing stored observed_mu transport and the rejected approaches
SIGNATURE_CODEC_OBLIGATIONS	SIGNATURE_CODEC_OBLIGATIONS.md	Actual 1474-byte layout, prefix proofs, and suffix/rANS obligations

Declaration or claim	File	What to check when reading
MODE2_HBZ_CODEC_ OBLIGATIONS	MODE2_HBZ_CODEC_ OBLIGATIONS.md	Statuses and success boundaries from Week 8 HBZ leaves/tables through the Week 14 production full wrapper
HBZ_FULL_WRAPPER_ COMPOSITION	HBZ_FULL_WRAPPER_ COMPOSITION.md	Week 14 production/focused direct-call surfaces, exact equivalence, and failure/success branches. The fixed all-zero failure exclusion left open here was closed by the Week 15 document and theorems
RANS_ACTUAL_SUCCESS_ WITNESS	RANS_ACTUAL_SUCCESS_ WITNESS.md	Design, verification scope, and losslessness boundary of the all-six normalization budget, actual failure-cause theorem, and production zero-HBZ success lift
RANS_PROOF_DESIGN	RANS_PROOF_DESIGN.md	Decomposition of the reverse normalization-byte stack and actual loop refinement, and the rejected monolithic approach
RANS_BYTE_STACK_ INVARIANT	RANS_BYTE_STACK_ INVARIANT.md	Canonical $xs/cuts/bytes$ semantics, byte order, two intended actual loop invariants, and non-vacuity boundary
RANS_ENCODER_ INVARIANT	RANS_ENCODER_ INVARIANT.md	Week 10 actual encoder suffix-index conventions and Week 11 tail-aware invariants, generated word updates, final serialization, and actual full-suffix closure
RANS_DECODER_ INVARIANT	RANS_DECODER_ INVARIANT.md	Week 12 reference states/cursors, outer symbol-prefix and tail frames, inner 0/1/2-byte replay invariants, initial parsing, and actual final-check boundaries
WEEK7_REPORT	WEEK7_REPORT.md	FREEZE-A/GO-B decision and historical 30-target baseline immediately before Week 8
WEEK8_REPORT	WEEK8_REPORT.md	CONTINUE-RANS verdict, HBZ leaves/tables/single steps, and historical 38-target boundary immediately before Week 9
WEEK9_REPORT	WEEK9_REPORT.md	Byte stack, actual suffix copy and harness control, two open outer-loop refinements, and historical 44-target boundary
WEEK10_REPORT	WEEK10_REPORT.md	Array/list and word bridges, actual inner bytes/frames, serialization leaves, successful sizes, and the 50-target CONTINUE-ENCODER boundary
Week 11 encoder trace note	latex/sections/ 27-week11-rans-encoder- trace-closure.tex	Research note explaining tail-aware invariants, generated word steps, final serialization, and actual outer-suffix closure, and separating the remaining success-witness/decoder boundaries

Declaration or claim	File	What to check when reading
WEEK11_REPORT	`WEEK11_REPORT.md`	Success-conditioned partial-correctness closure of `OBL-RANS-ENCODE-REFINEMENT`, 57 passing targets, the `GO-ENCODER` verdict, and the single Week 12 decoder obligation
Week 12 decoder refinement note	`latex/sections/28-week12-rans-decoder-refinement.tex`	Research note explaining actual decoder exact-trace inputs, cursor/table/replay invariants, actual final rejection checks, and the remaining composition edge
WEEK12_REPORT	`WEEK12_REPORT.md`	Exact-trace partial-correctness closure of `OBL-RANS-DECODE-REFINEMENT`, 65 passing targets, the `GO-DECODER` verdict, and the single Week 13 composition obligation
Week 13 rANS core composition note	`latex/sections/29-week13-rans-core-composition.tex`	Research note explaining copy traces, pointwise decoder reads, the W64 size bridge, and failure/success semantic composition of the actual harness module
WEEK13_REPORT	`WEEK13_REPORT.md`	Success-conditioned partial-correctness closure of `OBL-RANS-CORE-INVERSE`, 67 passing targets, the `GO-CORE` verdict, and the Week 14 actual full-HBZ wrapper obligation
Week 13 independent verification record	`logs/week13-independent-verifier.md`	Verdict independently cross-checking the final theorem's three actual procedure calls, non-circular preconditions, failure/success postconditions, and 67-target completion log
Week 14 full-HBZ composition note	`latex/sections/30-week14-hbz-full-wrapper-composition.tex`	Research note explaining full/focused internal equivalence, actual full encoder/decoder, the direct harness, and the production SignaturePack/Unpack lift
WEEK14_REPORT	`WEEK14_REPORT.md`	Success-conditioned partial-correctness closure of `OBL-SIG-HBZ-ENCODE-DECODE`, 72 passing targets, the `GO-HBZ` verdict, and the Week 15 all-zero success-witness obligation
Week 14 full-HBZ surface scan	`logs/week14-hbz-full-surface-scan.log`	Theorem-surface checks for production/focused direct calls, internal boundaries, preconditions, and failure/success frame conclusions
Week 15 actual-success note	`latex/sections/31-week15-rans-actual-success-witness.tex`	Research note explaining the all-six budget, failure-trace causes, actual rANS/full-HBZ/production zero-input success lift, and termination boundary

Declaration or claim	File	What to check when reading
WEEK15_REPORT	WEEK15_REPORT.md	Fixed all-six Hoare partial-correctness closure of **OBL-RANS-ACTUAL-SUCCESS-WITNESS**, 74 passing targets, and the GO-WITNESS verdict; not a losslessness verdict
Week 15 verification surfaces/completion logs	logs/week15-rans-success-surface-scan.log, logs/verify-all-week15.log	Checks that success/trace/capacity and losslessness are not preconditions and preserves the 74-target completion marker
Week 16 MINCORE plan/note	WEEK16_MINCORE_PLAN.md, latex/sections/32-week16-mincore-keygen-first-task.tex	Restricted KeyGen/Sign/Verify predicates, the decoded-object composition boundary, (h)-codec deferral, KeyGen/Sign stop decisions, and the active Verify frontier
WEEK16_KG_REPORT	WEEK16_KG_REPORT.md	Actual M23 matrix/finalizer snapshots, KG-2/adjusted KG-4 components, 76 passing targets, and the first-task verdict CONTINUE-KG/BLOCKED-KG-NTT-MUL; superseded by subsequent reports
Week 16 KG-NTT-MUL report/note	WEEK16_KG_NTT_MUL_REPORT.md, latex/sections/33-week16-kg-ntt-mul-stop.tex	Final output_row rewrite, missing odd-root convolution/security-list adapter, STOP-KG-NTT, and transition to the actual accepted Sign core
WEEK16_SIGN_REPORT	WEEK16_SIGN_REPORT.md	Direct calls to the three actual Sign helpers, accepted-branch control, 78-target completion, and STOP-SIGN-CHAL-MODE2; S-1–S-7 are unproved
WEEK16_VERIFY_REPORT	WEEK16_VERIFY_REPORT.md	Actual five-Verify-helper order, V-1/V-2/V-5/V-6, W64 norm/tail/mismatch, 82-target completion, and STOP-VERIFY-MATRIX-CRT
Week 16 verification surfaces/completion logs	logs/week16-keygen-core-surface-scan.log, logs/verify-all-before-kg-ntt-mul.log, logs/verify-all-week16-kg-ntt-mul.log	Checks the two actual calls, non-circular preconditions, and NTT boundary. The pre-change 76-target and continuation 77-target completion markers are in dedicated preserved logs, so the mutable summary is not used as historical 77-target evidence
Sign/Verify verification surfaces/completion logs	logs/week16-sign-core-surface-scan.log, logs/week16-verify-core-surface-scan.log, logs/verify-all-week16-verify-matrix-crt.log	Preserves Sign/Verify direct-call, non-circular precondition and stop surfaces, and the 82/82 aggregate immediately before final paper freeze
Paper-freeze artifact manifest	manifests/paper-artifacts.md	PAPER-FROZEN 82-target claim matrix, exact KG/Sign/Verify blockers, PDF/summary hashes, and reproduction procedure
Post-freeze NTT/Rq–HAETAE report	POST_FREEZE_KG_RQ_HAETAE_REPORT.md	Closure of pure NTT/Rq representation leaves, the native KeyGen row-product lift, and the first sampler-semantics blocker at that time

Declaration or claim	File	What to check when reading
Post-freeze actual avec/qj report	`POST_FREEZE_KG_ACTUAL_AVEC_QJ_REPORT.md`	Paper *a* from actual SHAKE128/rejection, separate paper *qj*, the synthetic security-object counterexample, and the latest `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS`
Current frozen-suite completion summary	`logs/verify-all-summary.txt`, `logs/verify-all-post-freeze-ntt-foundation.log`	Terminal `RESULTPASSauthored-targets=82cache=-no-eco`, including the paper-freeze audit; post-freeze auxiliary targets are not counted among the 82
Focused extraction manifest	`manifests/focused-extraction.md`	Correspondence among Jasmin roots, generated files, hashes, authored theorems, and focused HBZ body identity
Generated certificate manifest	`manifests/generated-certificates.sha256`	Pinned hashes of the 512-word actual HBZ symbol-certificate generator/output

B Notation and abbreviations

Functional correctness (FC)
The goal that implementation results or distributions agree with the paper specification.

Raw/internal The internal message path that does not use context-length encoding. Verify selects the actual raw branch when `prelen` is less than 2^{63}.

Accepted path An execution whose actual Verify return value is 0. The current theorem proves that this event implies `tail_reached`, but does not claim that arbitrary signatures or legitimate Sign outputs take the accepted path.

Exact trace An authored mirror program with the same final memory/return value as the actual procedure. Ghost observations are meaningful only when actual control flow reaches the corresponding point.

Region-local A relation requiring byte equality only on the address regions actually read by a procedure, instead of whole-memory equality. Differences at other addresses do not affect the result.

Product/replay boundary
The unresolved semantic edge transporting a relational theorem about generated hash returns to stored ghost results of an already completed sequential trace. It is currently explicitly **DEFERRED**.

Signature prefix The first 1056 bytes of a 1474-byte mode-2 signature. It contains 32 bytes of challenge bits and 1024 bytes of low coefficients, excluding suffix metadata, rANS payloads, and padding.

HBZ The layer of 1024 high-bit coefficients compressed by rANS in the signature suffix. The mode-2 canonical range is $[-6, 7)$, and the prepare leaf maps it to the 13-symbol alphabet $0, \ldots, 12$.

rANS step An arithmetic step reversibly updating the state and symbol using a frequency interval. The development covers pure/reciprocal single steps and the normalization readback/state-bound traces connecting them; separate Week 11/12 theorems connect the actual encoder and decoder outer loops, respectively.

Normalization-byte stack
The relation in which the decoder consumes bytes from the front that the encoder emitted through a backward cursor while processing symbols in reverse order. Pure *xs*/*cuts*/*bytes* semantics and word arithmetic are closed. The accumulated outer-suffix bridge, including actual encoder inner writes, was closed in Week 11; actual decoder forward consumption under exact inputs was closed in Week 12. The edge sequentially composing these two theorems with the actual copy call was closed in Week 13.

Array/list bridge
The Week 10 layer connecting the first 1024 symbols of the actual array to a pure list and its suffixes, and providing segment-match and frame operations.

Segment match The predicate `segment_matches`, stating that a pure byte list is stored pointwise starting at a specified array offset.

Prefix frame The predicate `prefix_frame`, stating that array bytes below a specified offset are unchanged across execution.

Actual inner loop
The normalization loop of generated `_rans_encode`. In addition to the conservative Week 10 0..4 phase theorem, the Week 11 tail-aware invariant derives the exact normalization length 0/1/2 from the actual guard and identifies it with the outer accumulated trace.

Tail-aware invariant
A verified Week 11 invariant placing both the current symbol's normalization bytes and the already existing pure-trace tail in `segment_matches`, restoring the outer one-symbol recurrence after the inner loop.

Exact-trace decoder input
A Week 12 predicate bundling `trace_bytes` computed from canonical 1024 symbols, size, state fields, actual decoder tables, and every normalization-byte read into one functional representation contract. It includes neither success, output equality, nor post-states as premises.

Partial replay invariant
An inner invariant connecting the number k of bytes already consumed within one decoder-symbol segment, the cursor, and the reduced state to `decoder_replay_prefix`. The actual guard equals $k < |\mathsf{segment}|$, and segment length is 0, 1, or 2.

Actual rANS harness
A unary composition running actual suffix copy and decoder only when the actual encoder return *bad* is 0. The existing harness theorem proves only branches and decoder input fields. A separate Week 13 semantic theorem transports the copied segment to exact-trace input, proving non-execution on failure and a 1024-symbol round trip on success in the same harness.

Production full-HBZ harness
: The Week 14 harness calling actual `SignaturePackMode2Target.M._encode_hb_z1_full` and, only when the returned size is nonzero, actual `SignatureUnpackMode2Target.M._decode_hb_z1_full`. Its results/branch flags exactly match the focused full-HBZ harness, and it proves failure preservation or 1024-coefficient recovery on success for canonical inputs.

Fixed all-six success witness
: Week 15 Hoare partial correctness that arithmetically shows the trace of 1024 occurrences of symbol 6, obtained by preparing zero HBZ, fits in a 1024-byte buffer, and excludes zero-size failure of a terminating actual encoder by contradiction. It does not mean execution existence, termination, or phoare $= 1$.

Success-conditioned
: A theorem form that explicitly concludes actual $\mathsf{size} = 0$ failure or nonzero success, without assuming encoding success in a fixed buffer from canonical inputs alone.

KeyGen snapshot
: The Week 16 boundary recording `pre_bp`, returned by actual `_kp_m23_matrix` and retained before the finalizer call, together with subsequent actual finalizer low/high and adjusted s_2 semantics. It does not call `pre_bp` the security model's $A_{\text{gen}} s_{\text{gen}}$.

NTT multiplication bridge
: The paper-freeze `STOP-KG-NTT` extended through the final representation rewrite expanding `KeygenM23ArithmeticSpec.output_row` into a full-invNTT/pointwise-word expression. Post-freeze work proved odd-root/full-NTT spectral action, the native Rq row product, and the Rq–HAETAE coefficient-representation adapter. Security ExpandVecA and full mod-$2q$ packaging do not follow automatically.

Paper freeze
: The fixed boundary at commit `4bde120`, aligning 82 authored targets, the claim matrix, theorem graph, reproduction procedures, and paper PDF. Later auxiliary theorems do not retroactively extend frozen PDF claims.

Post-freeze auxiliary layer
: Subsequent results verified in the external NTT/Rq tree, strengthened existing KeyGen targets, and `post-freeze/` without increasing the frozen manifest count. The current verdict is `STOP-KG-AVEC-QJ`.

MINCORE
: The Week 16 minimum paper scope targeting only the paper's core KeyGen/Sign/Verify equations and restricted decoded-object composition, excluding the full public APIs, codecs, sampler/retry, and termination. KeyGen is partial/stopped; Sign is partial/stopped at actual-call control; Verify is partial with helper-local word/control results retained; restricted composition remains a subsequent specification.

Generated procedure
: An EasyCrypt procedure generated from pinned Jasmin sources by `jasmin2ec`.

Wrapper A local module authored for proof alignment. Current central theorems place actual generated procedures, rather than wrappers, on the conclusion surface.

Zero loss There is no separate statistical-distance term because the two executions at the deterministic seam yield exactly the same result. It does not mean zero loss for the full API.

Reachability witness
A theorem showing that a relational theorem's premises are consistent and concretely constructed from preceding procedure returns.

TCB Trusted computing base. Currently includes EasyCrypt/Why3/provers, the Jasmin translator, pinned sources, and host verification tools.

Current research frontier

Direct comparison of `observed_mu` in accepted Sign/Verify traces was frozen as a paper boundary because there is no product/replay rule transporting pRHL hash returns to ghost fields of completed traces, not because necessary premises are missing. On the functional path, after Week 11 actual encoder successful-suffix refinement, Week 12 directly closed actual decoder initial parsing, table lookups, 0/1/2-byte replay, and outer/inner loops. A terminating actual decoder on exact `trace_bytes` input satisfies $bad = 0$, exact consumed size, 1024-symbol recovery, and the output-tail frame. Week 13 transported actual encoder `segment_matches` and copy-helper `slice_eq` to those pointwise inputs, closing the actual core inverse as success-conditioned partial correctness. Week 14 transported full/focused internal procedure equivalence, the actual full encoder/decoder, and the direct harness to the production SignaturePack/Unpack boundary, closing the full-HBZ wrapper inverse at the same strength of partial correctness. Week 15 composed the all-six trace budget with actual failure causes, transporting nonzero-size success for fixed zero HBZ to the production boundary. This is a Hoare theorem about terminating executions, however, not losslessness.

Week 16 composed the two actual M23 matrix/finalizer calls to close snapshot low/high, adjusted s_2, and the snapshot-only mod-$2q$ equation. The 77th manifested target then individually compiled theorems expanding `output_row` into the existing full-invNTT/pointwise-word representation and applying it to the actual snapshot's two active rows. The 82 surfaces, including restricted Sign/Verify results, are `PAPER-FROZEN`; their historical stopping points are `STOP-KG-NTT`, `STOP-SIGN-CHAL-MODE2`, and `STOP-VERIFY-MATRIX-CRT`.

Post-freeze work closed pure NTT spectral action, mathematical KeyGen/Verify row products, and the Rq–HAETAE representation adapter. Actual mode-2 `avec` was connected to paper a, and the paper qj correction was composed with the snapshot. However, compiled counterexamples confirmed that the existing security-side synthetic object differs from actual a and paper qj, so the current verdict is `STOP-KG-AVEC-QJ`. The first repair obligation is `OBL-KG-SECURITY-EXPANDVECA-SEMANTICS`, followed by full mod-$2q$ width-6 packaging. The pure NTT foundation for Verify is also closed, but actual 2-by-4 accumulation and CRT/freeze procedure leaves remain. The (h) codec, restricted composition, termination, context, highbits/LSB, probability distributions, and game-based security also remain separate obligations. The current frozen-suite 82/82 aggregate and paper-freeze audit are preserved in `logs/verify-all-summary.txt`.

References

[1] CryptoLab. Existing HAETAE easycrypt proof artifacts, 2026. Local source roots reused only through premise-preserving import adapters.

[2] CryptoLab. HAETAE specification, 2026. Pinned local artifact CryptoLab_HAETAE.pdf, 46 pages, produced 2026-02-04; SHA-256 80d28b3af9af2a27dba0eb4746f9b5755e05cb9a86bb6596f45fda06c605f3f6.

[3] CryptoLab. Pinned HAETAE jasmin implementation tree, 2026. Local source root haetae-ref-jasmin/ used for focused jasmin2ec extraction.